

OSCAT

Network:LIBRARY

Dokumentation

Version 1.0



Inhaltsverzeichnis

1. Geräte Treiber.....	4
1.1. <u>IRTRANS</u>	4
1.2. <u>IRTRANS_1</u>	4
1.3. <u>IRTRANS_4</u>	5
1.4. <u>IRTRANS_8</u>	6
1.5. <u>IRTRANS_DECODE</u>	7
2. Netzwerk und Kommunikation.....	9
2.1. <u>DNS_CLIENT</u>	9
2.2. <u>GET_WAN_IP</u>	10
2.3. <u>HTML_DECODE</u>	12
2.4. <u>HTML_ENCODE</u>	12
2.5. <u>HTTP_GET</u>	13
2.6. <u>IP4_CHECK</u>	15
2.7. <u>IP4_DECODE</u>	16
2.8. <u>IP4_TO_STRING</u>	16
2.9. <u>IP2GEO</u>	17
2.10. <u>IS_IP4</u>	18
2.11. <u>IS_URLCHR</u>	19
2.12. <u>IP_CONTROL</u>	19
2.13. <u>IP_FIFO</u>	25
2.14. <u>IP_CONTROL2</u>	27
2.15. <u>LOG_MSG</u>	28
2.16. <u>MB_CLIENT (OPEN-MODBUS)</u>	29
2.17. <u>MB_SERVER (OPEN-MODBUS)</u>	33
2.18. <u>MB_VMAP</u>	35
2.19. <u>PRINT_SF</u>	39
2.20. <u>READ_HTTP</u>	40
2.21. <u>SNTP_CLIENT</u>	41
2.22. <u>SNTP_SERVER</u>	42
2.23. <u>SPIDER_ACCESS</u>	43
2.24. <u>STRING_TO_URL</u>	45
2.25. <u>SYS_LOG</u>	46
2.26. <u>TELNET_LOG</u>	49
2.27. <u>TELNET_PRINT</u>	50
2.28. <u>URL_DECODE</u>	54
2.29. <u>URL_ENCODE</u>	54
2.30. <u>URL_TO_STRING</u>	54
2.31. <u>XML_READER</u>	55
2.32. <u>YAHOO_WEATHER</u>	61

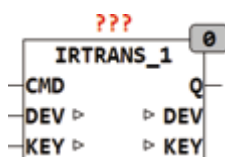
1. Geräte Treiber

1.1. IRTRANS

Die Bausteine IRTRANS_? stellen ein Interface für Infrarot Transmitter der Firma IRTrans GmbH zur Verfügung. IRTrans bietet Transmitter für RS232 und TCP/IP an, welche alle mit den folgenden Treiberbausteinen betrieben werden können. Der grundsätzliche Anschluss an RS232 oder TCP/IP hat mit den entsprechenden Herstellerrouitinen zu erfolgen. Die Interfacebausteine setzen auf ein Buffer Interface auf welches in einen Buffer (Array of Byte) die Daten und in einem Counter die Länge des Datenpakets in Bytes zur Verfügung stellt. Die IRTrans Geräte erlernen beliebige IR Tastencodes und setzen diese mittels einer konfigurierbaren Datenbank in ASCII Strings um. Mit der Ethernet Variante werden diese Strings dann über UDP versandt und können von einer SPS empfangen und ausgewertet werden. So können zum Beispiel vollautomatisch die Jalousien heruntergefahren werden wenn jemand den Fernseher einschaltet ohne das dafür eine zusätzliche Bedienung nötig wäre. Die SPS kann über Diesen Weg beliebig vielen Fernsteuerungen in unterschiedlichen Räumen zuhören und entsprechende Aktionen daraus ableiten. Umgekehrt ist natürlich auch die Aussendung von Tastencodes über die Transmitter Module möglich.

1.2. IRTRANS_1

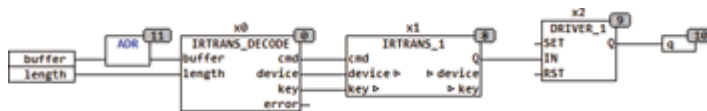
Type	Funktionsbaustein
Input	CMD : BOOL (TRUE wenn Daten zum Auswerten Anliegen) DEV : STRING (Name der Fernsteuerung) KEY : STRING (NAME der Taste)
Setup	DEV_CODE : STRING (zu dekodierender Fernsteuerungsname) KEY_CODE : STRING (zu dekodierender Tastencode)
Output	Q : BOOL (Ausgangssignal)



IRTRANS_1 überprüft wenn CMD = TRUE ob die Zeichenkette am Eingang

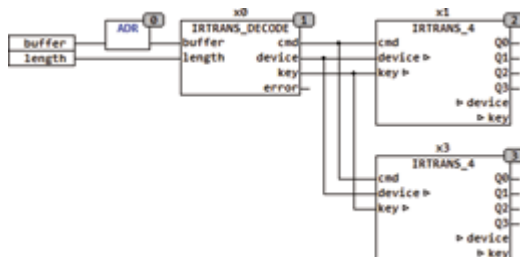
DEV dem DEV_CODE (Device Code) entspricht und die Zeichenkette am Eingang KEY dem KEY_CODE entspricht. Wenn die Codes übereinstimmen und CMD = TRUE ist dann wird der Ausgang Q für einen Zyklus auf TRUE gesetzt.

Das folgende Beispiel zeigt die Anwendung von IRTRANS_1:



In diesem Beispiel berechnet ADR() die Adresse des Datenpuffers und gibt sie zusammen mit der Länge an IRTRANS_DECODE. Der Decoder ermittelt aus gültigen Datenpaketen den String DEV und KEY und reicht diese per CMD an IRTRANS_1. IRTRANS_1 oder alternativ auch IRTRANS_4 und IRTRANS_8 überprüft dann ob DEV und KEY übereinstimmen und schaltet dann den Ausgang Q für einen Zyklus auf TRUE. Im Beispiel wird damit ein DRIVER_1 gesteuert, der es der Fernsteuerung erlaubt, mit jedem empfangenen Protokoll den Ausgang umzuschalten.

Wenn mehrere Key Codes ausgewertet werden sollen, können alternativ die Bausteine IRTRANS_4 oder IRTRANS_8 verwendet werden oder mehrere dieser Bausteine parallel betrieben werden.



1.3. IRTRANS_4

Type Funktionsbaustein

Input CMD : BOOL (TRUE wenn Daten zum Auswerten Anliegen)

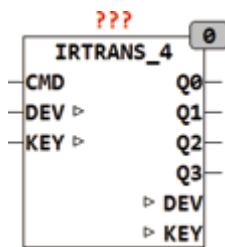
DEV : STRING (Name der Fernsteuerung)

KEY : STRING (NAME der Taste)

Setup DEV_CODE : STRING (zu dekodierender Fernsteuerungsname)

KEY_CODE_0..7 : STRING (zu dekodierender Tastencode)

Output Q : BOOL (Ausgangssignal)



IRTRANS_4 überprüft wenn CMD = TRUE ob die Zeichenkette am Eingang DEV dem DEV_CODE (Device Code) entspricht und die Zeichenkette am Eingang KEY einem KEY_CODE entspricht. Wenn die Codes übereinstimmen und CMD = TRUE ist dann wird der entsprechende Ausgang Q für einen Zyklus auf TRUE gesetzt. Weitergehende Informationen zur Funktion des Bausteins befinden sich unter IRTRANS_1.

1.4. IRTRANS_8

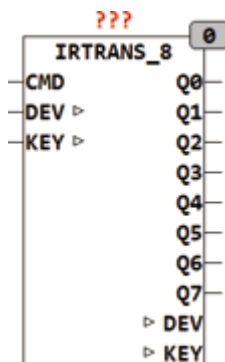
Type Funktionsbaustein

Input CMD : BOOL (TRUE wenn Daten zum Auswerten Anliegen)

I/O DEV : STRING (Name der Fernsteuerung)
KEY : STRING (NAME der Taste)

Setup DEV_CODE : STRING (zu dekodierender Fernsteuerungsname)
KEY_CODE_0..7 : STRING (zu dekodierender Tastencode)

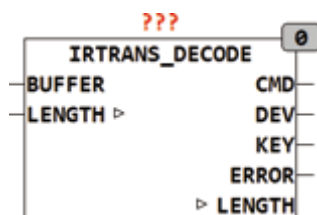
Output Q0..Q7 : BOOL (Ausgangssignal)



IRTRANS_8 überprüft wenn CMD = TRUE ob die Zeichenkette am Eingang DEV dem DEV_CODE (Device Code) entspricht und die Zeichenkette am Eingang KEY einem KEY_CODE entspricht. Wenn die Codes übereinstimmen und CMD = TRUE ist dann wird der entsprechende Ausgang Q für einen Zyklus auf TRUE gesetzt. Weitergehende Informationen zur Funktion des Bausteins befinden sich unter IRTRANS_1.

1.5. IRTANS_DECODE

Type	Funktionsbaustein
Input	BUFFER : Pointer to ARRAY of BYTE (Datenpuffer)
I/O	LENGTH : INT (Länge der Daten im Puffer)
Output	CMD : BOOL (TRUE wenn gültige Daten am Ausgang anliegen)
	DEV : STRING (Name der Fernsteuerung)
	KEY : STRING (Name des Tastencodes)
	ERROR : BOOL (TRUE wenn ein ungültiges Datenpaket vorliegt)



IRTRANS_DECODE empfängt die in BUFFER vorliegenden Daten mit der Länge LENGTH, überprüft ob ein gültiges Datenpaket vorliegt und dekodiert aus dem Datenpaket den Namen der Fernsteuerung und den Namen der Taste. Wenn ein gültiges Datenpaket dekodiert wurde liegt der Name der Fernsteuerung am Ausgang DEV und der Name der Taste am Ausgang KEY. Der Ausgang CMD signalisiert das neue Ausgangsdaten vorliegen. Der Ausgang ERROR wird dann gesetzt wenn ein Datenpaket empfangen wurde das nicht dem Format entspricht. Der Eingang BUFFER ist als Pointer ausgeführt um mit unterschiedlich großen Eingangspuffern arbeiten zu können. Die Adresse des BUFFERS kann mit der Funktion ADR() ermittelt werden.

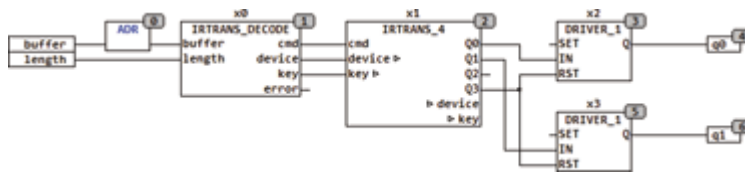
Das Format ist wie folgt definiert:

'Name der Fernsteuerung','Name des Tastencodes'\$R\$N

Ein Datenpaket besteht aus dem Namen der Fernsteuerung, gefolgt von einem Komma und anschließend der Name des Tastencodes. Das Datenpaket wird mit einem Carriage Return und einem Line Feed abgeschlossen.

Damit IRTANS_DECODE funktioniert muss in der IRTrans Konfiguration die Check box BROADCAST IR RELAY angekreuzt sein und in der entsprechenden Device Datenbank unter DEFAULT ACTION muss der String '%r,%c\r\n' eingetragen sein. IRTANS_DECODE wertet genau diesen String aus und dekodiert %r als Name der Fernbedienung und %c als die gedrückte Taste.

Das folgende Beispiel zeigt die Anwendung von IRTANS_DECODE mit einem UDP Server:



Der UDP Server liefert das Empfangene Datenpaket im RECEIVE_BUFFER ab und hinterlegt in DIREC_COUNT die Länge der Empfangsdaten in Bytes. IRTRANS_DECODE wird aktiv wenn DIREC_COUNT > 0 wird und überprüft dann ob die Empfangsdaten dem Format eines IRTRANS Empfangspakets entsprechen. Sind die Daten in RECEIVE_BUFFER gültige Empfangsdaten wird der Name der Fernsteuerung in DEV und der Tastencode in KEY abgespeichert, DIREC_COUNT auf 0 gesetzt und der Ausgang CMD für einen Zyklus auf TRUE gesetzt. Wenn die Eingangsdaten nicht dem vorgeschriebenen Format entsprechen wird ERROR auf TRUE gesetzt und die Länge in DIREC_COUNT wird nicht auf 0 gesetzt damit gegebenenfalls andere Decoder parallel die Daten im RECEIVE_BUFFER auswerten können. In der IRTRANS Web Konfiguration muss als Broadcast Adresse die IP Adresse der SPS eingetragen werden.

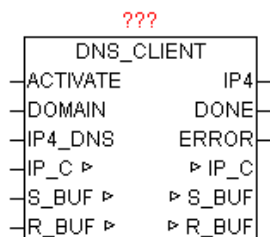
IRTRANS Web Konfiguration:

	LED D ▾
	☒ Broadcast IR Relay
UDP Broadcast Target	192.168.124.1
UDP Broadcast Port	21000
Store Settings	

2. Netzwerk und Kommunikation

2.1. DNS_CLIENT

Type	Funktionsbaustein
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
INPUT	ACTIVATE: BOOL (Abfrage starten durch positive Flanke) DOMAIN: STRING (Domain Name oder IP als String) IP4_DNS: DWORD (IPv4 Adresse des DNS-Server)
OUTPUT	IP4: DWORD (IPv4 Adresse der angefragten Domain) DONE: BOOL (IP der Domain erfolgreich ermittelt) ERROR: DWORD (Fehlercode)



DNS_CLIENT ermittelt aus den übergebenen qualifizierten DOMAIN Namen die zugehörige IPv4 Adresse z.B. „www.oscat.de“. Dazu wird eine DNS-Abfrage über den parametrierten DOMAIN Namen bei einen DNS-Server gemacht. Bei positiver Flanke von ACTIVATE wird die angegebene DOMAIN zwischen gespeichert, so dass diese nicht länger vorhanden sein muss. Sollte die Abfrage mehrere IP-Adressen liefern, so wird immer die letzte IP herangezogen. Als IP4_DNS kann ein beliebiger öffentlicher DNS-Server verwendet werden. Wenn die SPS hinter einen DSL-Router sitzt, kann dieser über seine Gateway-Adresse dieser als DNS-Server verwendet werden. Was letztendlich auch bei wiederkehrenden Abfragen zu schnelleren Antwortzeiten führt, da diese im Router-Cache verwaltet werden. Bei positiven Ergebnis DONE = TRUE enthält IP4 die gesuchte IP-Adresse, bis zum Start der nächsten Abfrage durch positive Flanke von ACTIVATE. Wird im DOMAIN Name eine gültige IPv4 Adresse erkannt, wird logischerweise keine DNS-Abfrage mehr durchgeführt und gleich in konvertierter Form wieder bei IPv4 ausgegeben und DONE auf TRUE gesetzt. ERROR liefert im Fehlerfall die genaue Ursache.

Fehlercodes:

Wert				Ursprung	Beschreibung
B3	B2	B1	B0		
nn	nn	nn	xx	IP_CONTROL	Fehler vom Baustein IP_CONTROL
xx	xx	xx	00	DNS_CLIENT	No error: The request completed successfully
xx	xx	xx	01	DNS_CLIENT	Format error: The name server was unable to interpret the query.
xx	xx	xx	02	DNS_CLIENT	Server failure: The name server was unable to process this query due to a problem with the name server.
xx	xx	xx	03	DNS_CLIENT	Name Error: Meaningful only for responses from an authoritative name server, this code signifies that the domain name referenced in the query does not exist
xx	xx	xx	04	DNS_CLIENT	Not Implemented: The name server does not support the requested kind of query
xx	xx	xx	05	DNS_CLIENT	Refused: The name server refuses to perform the specified operation for policy reasons
xx	xx	xx	06	DNS_CLIENT	YXDomain: Name Exists when it should not
xx	xx	xx	07	DNS_CLIENT	YXRRSet. RR: Set Exists when it should not
xx	xx	xx	08	DNS_CLIENT	NXRRSet. RR: Set that should exist does not
xx	xx	xx	09	DNS_CLIENT	Server Not Authoritative for zone
xx	xx	xx	0A	DNS_CLIENT	Name not contained in zone

2.2. GET_WAN_IP

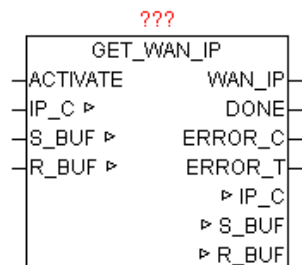
Type Funktionsbaustein:

IN_OUT IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)
 S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten)
 R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
 INPUT ACTIVATE: BOOL (Freigabe zur Abfrage)
 OUTPUT: WAN_IP: DWORD (Wide-Area-Network-Adresse)

DONE: BOOL (Abfrage ohne Fehler beendet)

ERROR_C: DWORD (Fehlercode)

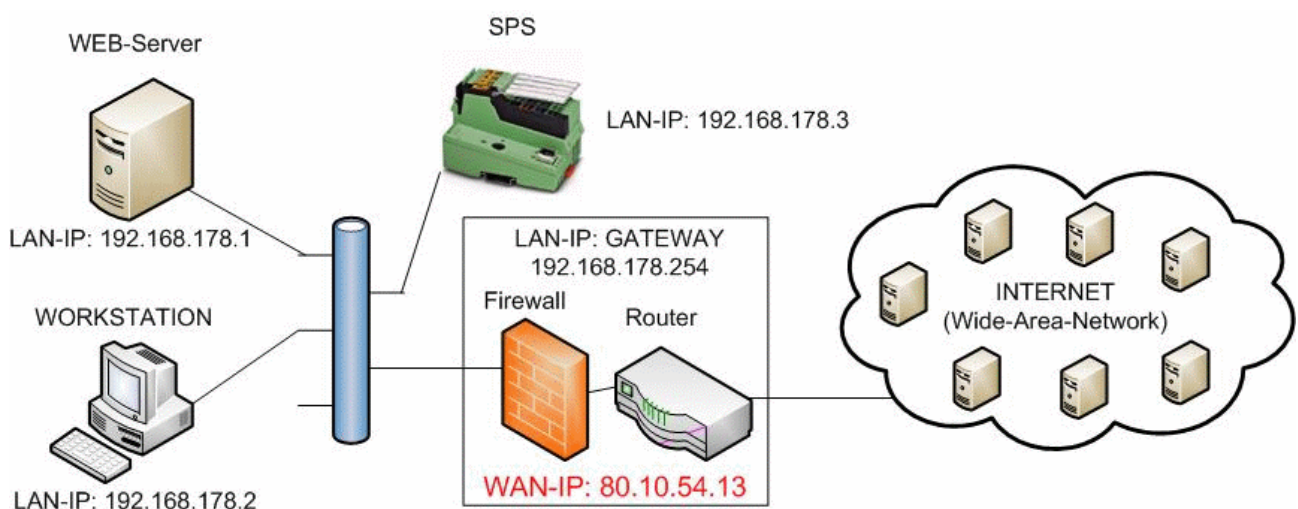
ERROR_T: BYTE (Fehlertype)



Der Baustein stellt die IP-Adresse fest, die der Internet-Routers im Wide-Area-Network (Internet) benutzt. Diese IP-Adresse ist z.B. notwendig um DynDNS Anmeldungen durchführen zu können. Mit einer positiven Flanke von ACTIVATE wird die Abfrage gestartet. Nach erfolgreich beendeter Abfrage wird DONE=TRUE ausgegeben, und bei Parameter WAN_IP wird die aktuelle WAN-IP Adresse ausgegeben. Sollte bei der Abfrage ein Fehler auftreten so wird dieser unter ERROR_C gemeldet in Kombination mit ERROR_T.

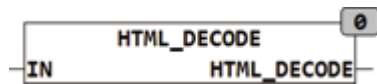
ERROR_T:

Wert	Eigenschaften
1	Die genaue Bedeutung von ERROR_C ist beim Baustein DNS_CLIENT nachzulesen
2	Die genaue Bedeutung von ERROR_C ist beim Baustein HTTP_GET nachzulesen



2.3. HTML_DECODE

Type Funktion : STRING(255)
 Input IN : STRING (Zeichenkette)
 Output STRING(255) (Zeichenkette)



HTML_DECODE wandelt reservierte Zeichen welche in der Form &name; im HTML Code gespeichert sind in die originalen Zeichen um. Zusätzlich werden alle codierten Sonderzeichen in den entsprechenden ASCII Code umgewandelt. Sonderzeichen können in HTML durch folgende Zeichenfolge repräsentiert sein:

- &#NN; wobei NN die Position des Zeichens innerhalb der Zeichentabelle in dezimaler Schreibweise darstellt.

- &#xNN; oder &#XNN wobei NN die Position des Zeichens innerhalb der Zeichentabelle in hexadezimaler Schreibweise darstellt.

&Name; Sonderzeichen haben Namen wie zum Beispiel € für €.

Die reservierten Zeichen in HTML sind:

& wird codiert als &

> wird codiert als >

< wird codiert als <

" wird codiert als "

Beispiele:

HTML_DECODE('1 ist >als 0') = '1 ist > als 0';

HTML_DECODE('&#D79;&#D83;&#D67;&#D65;&#D84;') = 'OSCAT';

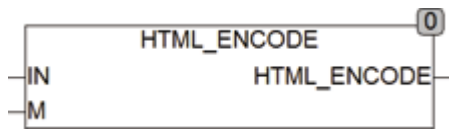
HTML_DECODE('&#xH4F;&#xH53;&#xH43;&#xH41;&#xH54;') = 'OSCAT';

HTML_DECODE('&#XH4F;&#XH53;&#XH43;&#XH41;&#XH54;') = 'OSCAT';

2.4. HTML_ENCODE

Type Funktion : STRING(255)
 Input IN : STRING (Zeichenkette)
 M : BOOL (Mode)

Output STRING(255) (Zeichenkette)



HTML_ENCODE wandelt in HTML reservierte Zeichen um in die Form &Name;. Wird der Eingang M auf TRUE gesetzt werden zusätzlich alle Zeichen mit dem Code 160-255 und 128 in die &Name Konvention umgesetzt.

Vorsicht ist bei der Anwendung von Zeichensätzen geboten weil diese nicht auf allen Systemen identisch sind und Abweichungen speziell bei Sonderzeichen häufig vorkommen. So ist zum Beispiel nicht bei allen Systemen das € Zeichen auf Position 128 in der Zeichentabelle.

Die reservierten Zeichen in HTML sind:

& wird codiert als &

> wird codiert als >

< wird codiert als <

" wird codiert als "

HTML_ENCODE wandelt die Zeichenkette '1 ist > als 0' um in '1 ist > als 0'.

2.5. HTTP_GET

Type Funktionsbaustein:

IN_OUT URL_DATA : Datenstruktur 'URL' (Daten von STRING_TO_URL)

IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)

S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten)

R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)

INPUT IP4: DWORD (IP-Adresse des HTTP-Servers)

GET: BOOL (Startet die HTTP Abfrage)

MODE: BYTE (Version der HTTP-GET Abfrage)

UNLOCK_BUF: BOOL (Freigabe des Empfangs-Datenbuffers)

OUTPUT HTTP_STATUS: STRING (ermittelter HTTP Statuscode)

HTTP_START: UINT (Start-Position des Message-Headers)

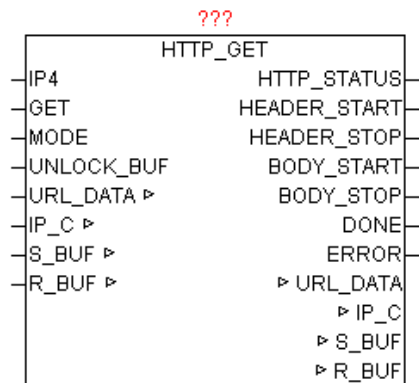
HTTP_STOP: UINT (Stopp-Position des Message-Headers)

BODY_START: UINT (Start-Position des Message-Body)

BODY_STOP: UINT (Stopp-Position des Message-Body)

DONE: BOOL (Aufgabe ohne Fehler durchgeführt)

ERROR: DWORD (Fehlercode)



HTTP_GET führt nach positiver Flanke von GET ein GET-Kommando auf einem HTTP-Server durch. Mittels MODE kann die HTTP Protokollversion vorgegeben werden. Die gewünschte URL (Web-Link) muss vorab in der URL_DATA Struktur fertig aufbereitet vorliegen. Die vollständige URL sollte darum vor dem Baustein aufruf mittels „STRING_TO_URL“ aufbereitet werden. Nach erfolgreicher Abfrage wird DONE = TRUE ausgegeben, und die Parameter HTTP_START und HTTP_STOP zeigen auf den Datenbereich in dem die Message-Header Daten zur weiteren Bearbeitung und Auswertung vorzufinden sind. Im Normalfall ist auch ein Message-Body vorhanden, der wiederum mittels BODY_START und BODY_STOP übermittelt wird. Weiters wird auf HTTP_STATUS der zurückgemeldete HTTP-Statuscode als String ausgegeben. Eine der Schwierigkeiten beim HTTP Empfang ist das erkennen vom Ende des Datenstream. Dabei werden vom Baustein mehrere Strategien verfolgt. Bei Anwendung von HTTP/1.0 wird das Ende des Datenempfangs über einen Verbindungsabbau seitens des Host erkannt. Weiters wird immer geprüft ob sich im Header die Info „Content-Length“ befindet, damit kann dann eindeutig erkannt werden ob alle Daten empfangen wurden. Trifft keines der vorigen Varianten zu, so wird über einen einfachen Receive-Timeout Error das Ende der Datenübertragung festgestellt und beendet. Einziger Nachteil dabei ist, das dies je nach eingestellte Timeout Zeit mitunter länger dauert als gewünscht. Darum ist es nicht schlecht wenn eine vernünftiger Timeout-Wert am IP_CONTROL gewählt wird. ERROR liefert im Fehlerfall die genaue Ursache (Siehe Baustein IP_CONTROL)

Folgende MODE können verwendet werden:

Mode	Protokollversion	Eigenschaften
------	------------------	---------------

0	HTTP/1.0	Der Host beendet selbstständig nach Übertragung der Daten die TCP-Verbindung
1	HTTP/1.0	Durch Anwendung von „Connection: Keep-Alive“ wird der Host angewiesen eine persistente Verbindung zu verwenden. Der Client sollte nach Ende der Aktivitäten die Verbindung wieder abbauen.
2	HTTP/1.1	Der Host verwendet eine persistente Verbindung und muss von Client beendet werden.
3	HTTP/1.1	Durch Anwendung von „Connection: Close“ wird der Host angewiesen nach Übertragung der Daten die TCP-Verbindung zu beenden.

2.6. IP4_CHECK

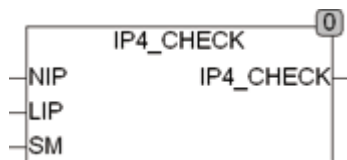
Type Funktion : BOOL

Input NIP : DWORD (Netzwerk IP Adresse)

LIP : DWORD (Lokale IP Adresse)

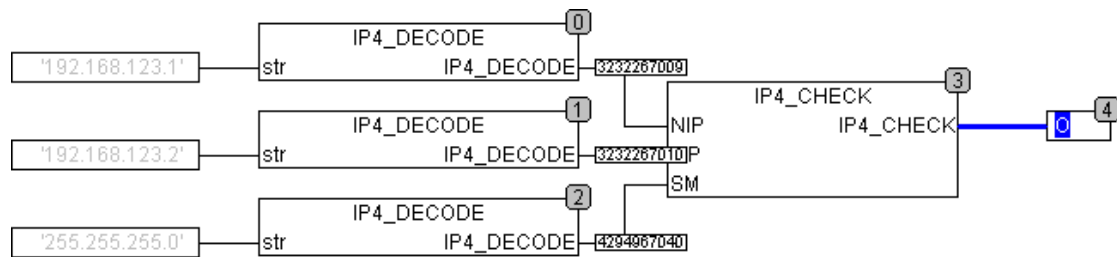
SM : DWORD (Subnet Maske)

Output BOOL (TRUE wenn NIP und LIP im gleichen Subnet liegen)



IP4_CHECK prüft ob eine Netzwerkadresse NIP und die Lokale Adresse LIP im gleichen Subnet liegen. Beiden Adressen werden zuerst mit der Subnet Maske Maskiert und dann auf Gleichheit geprüft. Es werden nur die Bits auf Gleichheit geprüft die in der Subnet Maske TRUE sind. Die Netzwerk Adressen müssen dem Ipv4 Format entsprechen und als DWORD vorliegen. Sollen IP Adressen die als String vorliegen geprüft werden müssen diese vorher in DWORD gewandelt werden.

Folgendes Beispiel zeigt 2 IP Adressen und eine Subnet Maske die als String vorliegen und nach entsprechender Umwandlung in DWORD geprüft werden. Der Ausgang wird TRUE weil beide Adressen im gleichen Subnet liegen.



2.7. IP4_DECODE

Type Funktion : DWORD

Input STR : BOOL (Zeichenkette die die IP Adresse enthält)

Output DWORD (dekodierte IP v4 Adresse)



IP4_DECODE dekodiert die in STR enthaltene Zeichenkette als IP v4 Adresse und gibt diese als DWORD zurück. Eine Rückgabe von 0 bedeutet das eine ungültige Adresse oder die Adresse '0.0.0.0' ausgewertet wurde. IP4 kann auch zum Auswerten einer Subnet Maske des IP v4 Formates verwendet werden.

2.8. IP4_TO_STRING

Type Funktion : STRING(15)

Input IP4 : BOOL (Zeichenkette die die IP Adresse enthält)

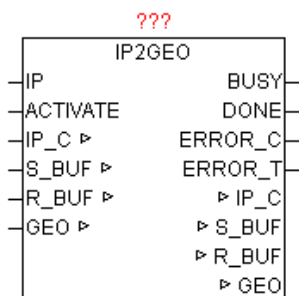
Output DWORD (dekodierte IP v4 Adresse)



IP4_TO_STRING wandelt die als DWORD gespeicherte IP4 Adresse in IN in einen STRING um. Das Format entspricht 'NNN.NNN.NNN.NNN'.

2.9. IP2GEO

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten) GEO: Datenstruktur 'IP2GEO' (Geographische Daten)
INPUT	ACTIVATE: BOOL (Freigabe zur Abfrage)
OUTPUT	BUSY: BOOL (Abfrage ist aktiv) DONE: BOOL (Abfrage ohne Fehler beendet) ERROR_C: DWORD (Fehlercode) ERROR_T: BYTE (Fehlertype)



Der Baustein liefert aufgrund der Wide-Area-Network IP-Adresse die Geographischen Informationen des Internet-Zuganges. Da die WAN-IP-Adressen weltweit registriert sind, lässt sich somit annähernd die Geographische Position der SPS bestimmen. Sollte der Zugang über einen Proxy-Server laufen, so wird die Geographische Position von diesen ermittelt und nicht von der SPS. Normalerweise ist der aber in unmittelbarer Nähe der SPS, und somit die Abweichung nicht relevant. Somit ergeben sich ermittelte Positionen die nur ein paar Kilometer von der wirklichen Position abweichen, und relativ genau sind.

Wird am Parameter „IP“ keine IP-Adresse angegeben, so wird automatisch die aktuelle WAN-IP herangezogen, andernfalls werden die Informationen der parametrierten IP geliefert. Mit einer positiven Flanke von ACTIVATE wird die Abfrage gestartet. Solange die Abfrage nicht beendet ist, wird BUSY=TRUE ausgegeben. Nach erfolgreich beendeter Abfrage wird DONE=TRUE ausgegeben, und bei Parameter WAN_IP wird die aktuelle WAN-IP Adresse ausgegeben. Sollte bei der Abfrage ein Fehler auftreten so wird dieser unter ERROR_C gemeldet in Kombination mit ERROR_T.

ERROR_T:

Wert	Eigenschaften
1	Die genaue Bedeutung von ERROR_C ist beim Baustein DNS_CLIENT nachzulesen
2	Die genaue Bedeutung von ERROR_C ist beim Baustein HTTP_GET nachzulesen

IP2GEO Datenstruktur:

Name	Typ	Eigenschaften
STATE	BOOL	Daten sind gültig
IP4	DWORD	IP-Adresse der geographischen Daten
COUNTRY_CODE	STRING(2)	Code des Landes (ISO Format) z.B. AT = Austria
COUNTY_NAME	STRING(20)	Name des Landes
REGION_CODE	STRING(2)	Code der Region (FIPS Format) z.B. 09 = Vienna
REGION_NAME	STRING(20)	Name der Region
CITY	STRING(20)	Name der Stadt
GEO_LATITUDE	REAL	Breitengrad des Ortes
GEO_LONGITUDE	REAL	Längengrad des Ortes
TIME_ZONE	INT	Geographische Zeitzone
UTC_OFFSET	REAL	Offset zu Universelle Weltzeit in Minuten

Der „COUNTRY_CODE ist nach „ISO 3166 Country-Code ALPHA-2“ kodiert.

http://www.iso.org/iso/english_country_names_and_code_elements

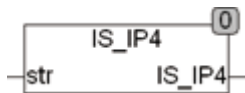
<http://de.wikipedia.org/wiki/ISO-3166-1-Kodierliste>

Der „REGION_CODE“ ist nach „FIPS Region Code“ kodiert.

http://en.wikipedia.org/wiki/List_of_FIPS_region_codes

2.10. IS_IP4

Type	Funktion : BOOL
Input	STR : STRING (zu prüfende Zeichenkette)
Output	BOOL (TRUE wenn STR eine gültige IP v4 Adresse enthält)



IS_IP4 prüft ob die Zeichenkette STR eine gültige IP v4 Adresse enthält, wenn nicht wird FALSE zurückgegeben. Ein gültige IP v4 Adresse besteht aus 4 Zahlen von 0 - 255 die mit je einem Punkt getrennt sind. Die Adresse 0.0.0.0 wird dabei als falsch eingestuft.

IS_IP4(0.0.0.0) = FALSE

IS_IP4(255.255.255.255) = TRUE

IS_IP4(256.255.255.255) = FALSE

IS_IP4(0.1.2.) = FALSE

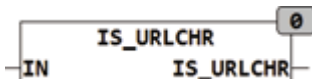
IS_IP4(0.1.2.3.) = FALSE

2.11. IS_URLCHR

Type Funktion : BOOL

Input IN : STRING (zu prüfende Zeichenkette)

Output BOOL (TRUE wenn STR eine gültige IP v4 Adresse enthält)



IS_URLCHR prüft ob die Zeichenkette nur zulässige Zeichen für eine URL Kodierung enthält. Enthält die Zeichenkette reservierte Zeichen gibt Sie FALSE zurück, andernfalls TRUE.

Für eine URL sind folgende Zeichen zulässig:

[A..Z]

[a..z]

[0..9]

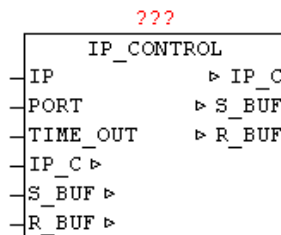
[-._~]

alle anderen Zeichen sind reserviert oder nicht zulässig.

2.12. IP_CONTROL

Type Funktionsbaustein

IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)
	S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten)
	R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
INPUT	IP: DWORD (kodierte IP Adresse als Standardadresse)
	PORT: WORD (Portnummer der IP-Adresse)
	TIME_OUT: TIME (Überwachungszeit)



Der IP_CONTROL ermöglicht die Hersteller und Plattform neutrale Nutzung der Ethernet-Kommunikation. Um die vielen verschiedene Schnittstellen der Steuerungslieferanten zu vereinen, wird der IP_CONTROL als Adapter „WRAPPER“ eingesetzt. Mit diesem Baustein können UDP und TCP sowie aktive und passive Verbindungen gehandhabt werden. Bei manchen Kleinststeuerungen ist auch die Anzahl an gleichzeitige geöffneten Sockets sehr begrenzt, darum unterstützt dieser Baustein auch das Sharing von Sockets. Durch eine integrierte automatische Koordinierung der Zugriffe ist es möglich das sich viele Client-Bausteine einen Socket teilen. Hierbei wird automatisch bei Zugriff erkannt ob ein Client andere Verbindungsparameter als sein Vorgänger benutzt. Eine bestehende Verbindung wird gegebenenfalls automatisch beendet und mit den neuen Verbindungsparametern wieder aufgebaut. Die Art der Verbindung kann mittels C_MODE eingestellt werden (Siehe Tabelle). Über C_PORT wird die gewünschte Port-Nummer vorgegeben, und mittels C_IP die IP v4 Adresse. Über C_STATE kann festgestellt werden ob die Verbindung auf - abgebaut ist , bzw. die negative und positive Flanke bei Änderung des Zustandes. Mittels C_ENABLE wird die Verbindung freigeben (aufgebaut) bzw. gesperrt (abgebaut). Das Daten senden und empfangen funktioniert unabhängig voneinander, das heißt es ist auch asynchrones Senden und Empfangen möglich wie z.B. bei Telnet. Bei Anwendungen bei denen nur gesendet wird, und kein Datenempfang erwartet wird, muss R_OBSERVE = FALSE sein, damit kein Timeout beim Empfang auftritt. Beim Beginn von Sende und Empfangsaktivitäten wird TIME_RESET vom Anwender einmalig auf TRUE gesetzt, damit alle Timeout mit einen definierten Startwert neu beginnen. Dies ist aufgrund der Sharing Funktionalität erforderlich, denn hier können aufgebaute Verbindungen bestehen bleiben und die Zugriffsrechte werden dabei nur weitergereicht. Der Parameter IP dient als mögliche Default-IP-Adresse. Um nicht mehrmals die gleiche IP-Adresse parametrieren zu müssen, kann eine Default - IP verwendet werden. Ein möglicher Einsatz wäre die Angabe der DNS-Server-Adresse. Wenn der

Baustein als C_IP die IP 0.0.0.0 erkennt, so wird automatisch die parametrierte IP Adresse verwendet. Das gleiche Verhalten besteht beim Parameter Port. Wird bei C_PORT der Port 0 erkannt so wird die am Baustein parametrierte Portnummer verwendet. Der Fehlercode ERROR setzt sich aus mehreren Teilen zusammen (Siehe ERROR Tabelle). Mittels TIMEOUT kann die allgemeine Überwachungszeit vorgeben werden. Dieser Zeitwert wird für Verbindungsaufbau, Daten senden und Daten empfangen jeweils unabhängig voneinander verwendet. Der übergebene TIMEOUT Zeitwert wird automatisch auf 200ms Minimum begrenzt. Somit kann dieser Parameter auch frei bleiben.

Wenn bei freigegebener Verbindung in den Sende buffer die Daten und eine Datenlänge eingetragen werden, wird automatisch der Datenblock versendet. Bei Datenempfang werden die Daten immer wieder zu den schon bestehenden Daten im Buffer angehängt. Durch eintragen von SIZE = 0 wird der Empfangs-Datenzeiger wieder zurückgesetzt, und die nächsten empfangenen Daten werden wieder ab Position 0 abgelegt.

Der Baustein unterstützt das Blocken der Datentelegramme, das heißt der S_BUF bzw. R_BUF kann beliebig groß sein. Einzelne empfangene Daten-Telegramme werden im R_BUF in Form eines Stream gesammelt. Genauso wird beim Senden von Daten verfahren. Die Daten im S_BUF werden in zulässiger Blockgröße einzeln als Stream versendet.

Anwendungs-Beispiel:

```

CASE state OF
00: (* auf Freigabe warten *)
  IF FREIGABE THEN
    state := 10;
    IP_STATE := 1; (* Anmelden *)
  END_IF;
10: (* Warten auf Zugriffsfreigabe zum Verbindung aufbauen und Datensenden *)
  IF IP_STATE = 3 THEN (* Zugriff erlaubt ? *)
    (* IP Datenverkehr einrichten *)
    IP_C.C_PORT := 123; (* Port-Nummer eintragen *)
    IP_C.C_IP := IP4; (* IP eintragen *)
    IP_C.C_MODE := 1; (* Mode: UDP+AKTIV+PORT+IP *)
    IP_C.C_ENABLE := TRUE; (* Verbindungsaufbau freigeben *)
    IP_C.TIME_RESET := TRUE; (* Zeitüberwachung rücksetzen *)
    IP_C.R_OBSERVE := TRUE; (* Datenempfang überwachen *)
    R_BUF.SIZE := 0; (* Empfangslänge rücksetzen *)
  
```

```

(* Sendedaten eintragen *)
S_BUF.BUFFER[0] := BYTE#16#1B;
(* usw.... *)
S_BUF.SIZE := xx; (* Sendelänge eintragen *)
state := 30;
30:
IF IP_C.ERROR <> 0 THEN
  (* Fehlerauswertung durchführen *)
  ELSIF S_BUF.SIZE = 0 AND R_BUF.SIZE >= xxx THEN
    (* Empfangene Daten auswerten *)

    (* Abmelden – Zugriff wieder für andere freigeben *)
    IP_STATE := BYTE#4;
    state := 00; (* Ablauf beenden *)
  END_IF;
END_CASE;

(* IP_FIFO Zyklisch aufrufen *)
IP_FIFO(FIFO:=IP_C.FIFO, STATE:=IP_STATE, ID:=IP_ID);
IP_C.FIFO:=IP_FIFO.FIFO;
IP_STATE := IP_FIFO.STATE;
IP_ID:=IP_FIFO.ID;

```

Die Datenstruktur IP_CONTROL enthält folgende Felder:

Datenfeld	Datentyp	Beschreibung
C_MODE	BYTE	Type der Verbindung
C_PORT	WORD	Port-Nummer
C_IP	DWORD	kodierte IP v4 Adresse
C_STATE	BYTE	Status der Verbindung
C_ENABLE	BOOL	Verbindungsfreigabe
R_OBSERVE	BOOL	Datenempfang überwachen
TIME_RESET	BOOL	Rücksetzen der Überwachungszeiten
ERROR	DWORD	Fehlercode
FIFO	IP_FIFO	Datenstruktur der Zugriffsverwaltung (kein Anwender-Zugriff notwendig)

Folgende C_MODE können verwendet werden:

TYP	TCP / UDP	Aktiv / Passiv	Port-Nummer erforderlich	IP-Adresse erforderlich
0	TCP	Aktiv	Ja	Ja
1	UDP	Aktiv	Ja	Ja
2	TCP	Passiv	Ja	Ja (Adresse des aktiven Partners)
3	UDP	Passiv	Ja	Ja (Adresse des aktiven Partners)
4	TCP	Passiv	Ja	Nein (beliebiger aktiver Partner wird akzeptiert)
5	UDP	Passiv	Ja	Nein (beliebiger aktiver Partner wird akzeptiert)

C_STATE:

Wert	Zustandsmeldung
0	Verbindung ist abgebaut
1	Verbindung wurde abgebaut (negative Flanke) Wert ist nur für einen Zyklus vorhanden, dann wird 0 ausgegeben
254	Verbindung wurde aufgebaut (positive Flanke) Wert ist für einen Zyklus vorhanden, dann wird 255 ausgegeben
255	Verbindung ist aufgebaut
<127	Abfrage ob Verbindung abgebaut ist
>127	Abfrage ob Verbindung aufgebaut ist

ERROR:

DWORD				Meldungstyp	Beschreibung
B3	B2	B1	B0		
00	xx	xx	xx	Verbindungsaufbau	Wert 00 - Keine Fehler vorhanden
nn	xx	xx	xx	Verbindungsaufbau	Wert 01 - 254 System spezifischer Fehler
FF	xx	xx	xx	Verbindungsaufbau	Wert 255 - Timeout Fehler
xx	00	xx	xx	Daten senden	Wert 00 - Keine Fehler vorhanden
xx	nn	xx	xx	Daten senden	Wert 01 - 254 System spezifischer Fehler

xx	FF	xx	xx	Daten senden	Wert 255 - Timeout Fehler
xx	xx	00	xx	Daten empfangen	Wert 00 - Keine Fehler vorhanden
xx	xx	nn	xx	Daten empfangen	Wert 01 - 254 System spezifischer Fehler
xx	xx	FF	xx	Daten empfangen	Wert 255 - Timeout Fehler
xx	xx	FE	xx	Daten empfangen	Wert 254 – Empfangsbuffer ist voll (Überlauf) (Buffer-Size wird automatisch auf 0 gesetzt)
xx	xx	xx	nn	Applikation-Error	Bei IP_CONTROL immer 00. ERROR wird von der Client-Applikation 1:1 übernommen und gegebenenfalls wird an dieser Stelle ein Applikation-Error eingetragen. Diese Fehlercode wird aber nur von den Client-Bausteinen eingetragen.

System spezifische Fehler: (PCWORX / MULTIPROG)

Wert	Zustandsmeldung
0x00	Kein Fehler aufgetreten.
0x01	Erstellen mindestens einer Task ist fehlgeschlagen.
0x02	Initialisierung des Socket-Interfaces fehlgeschlagen (nur WinNT).
0x03	Dynamischer Speicher konnte nicht reserviert werden.
0x04	FB kann nicht initialisiert werden, da beim Starten der asynchronen Kommunikations-Tasks ein Fehler aufgetreten ist.
0x10	Socket-Initialisierung fehlgeschlagen.
0x11	Fehler beim Senden einer Nachricht.
0x12	Fehler beim Empfang einer Nachricht.
0x13	Unbekannter Service-Code im Telegramm-Header.
0x21	Ungültiger Zustandsübergang beim Verbindungsaufbau.
0x30	Keine freien Kanäle mehr verfügbar.
0x31	Die Verbindung wurde abgebrochen.
0x33	Allgemeiner Timeout, Empfänger antwortet nicht mehr bzw. Sender hat Übertragung nicht beendet.
0x34	Verbindungswunsch wurde negativ quittiert.
0x35	Empfänger hat Sendung nicht bestätigt, evtl. ist Empfänger überlastet (Übertragung wiederholen).

0x40	Partner-String ist zu lang (255 Zeichen maximal).
0x41	Die angegebene IP-Adresse ist nicht gültig oder konnte nicht richtig interpretiert werden.
0x42	Nicht zulässige Port-Nummer.
0x45	Unbekannter Parameter an Eingang "PARTNER".
0x50	Sendeversuch auf ungültiger Verbindung (Sender oder Empfänger) .
0x53	Alle verfügbaren Verbindungen sind belegt.
0x61	Neg. Confirmation des Empfängers. Es wurde eine falsche Sequenznummer verwendet.
0x62	Datentyp von Sender und Empfänger sind ungleich.
0x63	Empfänger ist im Augenblick nicht empfangsbereit (mgl. Ursache: Empfänger ist disabled oder befindet sich gerade in der Datenübernahme (NDR=TRUE)).
0x64	Es wurde kein Empfangsbaustein mit der entsprechenden R_ID gefunden.
0x65	Eine andere Baustein-Instanz arbeitet bereits auf dieser Verbindung.
0x70	Partner-Steuerung wurde nicht als Time-Server konfiguriert.

System spezifischer Fehler: (CoDeSys)

0x00	Kein Fehler aufgetreten.
0x01	SysSockCreate nicht erfolgreich ausgeführt
0x02	SysSockBind nicht erfolgreich ausgeführt
0x03	SysSockListen nicht erfolgreich ausgeführt

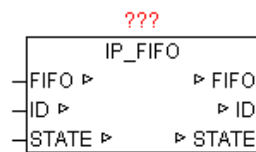
2.13. IP_FIFO

Type Funktionsbaustein:

IN_OUT FIFO : Datenstruktur 'IP_FIFO_DATA' (IP-FIFO Verwaltungsdaten)

ID: BYTE (aktuelle vom IP_FIFO-Baustein vergebene ID)

STATE: BYTE (Steuerbefehle und Statusmeldungen)



Dieser Baustein dient in Kombination mit IP_CONTROL zur Ressourcen-Verwaltung. Damit ist es möglich das Client-Applikation die alleinigen Zugriffsrechte anfordern und auch wieder abgeben können. Durch das FIFO wird gewährleistet das jeder Teilnehmer gleich oft den Ressourcen-Zugriff zugeteilt bekommt.

Bei ersten Aufruf des Bausteins wird automatisch eine neue eindeutige Applikation-ID vergeben, mittels der dann die Verwaltung im FIFO durchgeführt wird.

Der Parameter STATE wird von der Applikation als auch vom IP_FIFO Baustein verändert.

Jede Applikation kann sich in der Standardeinstellung nur einmal im FIFO eintragen.

Ablauf:

1. Applikation setzt STATE auf 1
2. Zugriffsrechte werden angefordert, während dessen ist STATE=2
3. wenn Ressource frei ist, und Zugriffsrechte vorhanden sind, dann ist STATE=3
4. Wenn die Applikation die Ressource bzw. den Zugriff nicht mehr benötigt, wird von der Applikation STATE auf 4 gesetzt. Danach gibt IP_FIFO die Ressource wieder frei und setzt STATE auf 0.
5. Vorgang wiederholt sich (gleicher oder anderer Applikation)

Anwendungsbeispiel ist beim Baustein IP_CONTROL zu finden !

Die Datenstruktur IP_FIFO_DATA enthält folgende Felder:

Datenfeld	Datentyp	Beschreibung
X	ARRAY [1..128] OF BYTE	FIFO-Speicher mit eingetragenen ID's
Y	ARRAY [1..128] OF BYTE	Anzahl der Einträge pro ID's
ID	BYTE	Zuletzt vergebene ID (Höchste ID)
MAX_ID	BYTE	Maximale Anzahl von Anmeldungen pro ID
INIT	BOOL	Initialisierung durchgeführt

EMPTY	BOOL	FIFO ist leer
FULL	BOOL	FIFO ist voll (sollte nicht passieren !)
TOP	INT	Maximale Anzahl an Einträgen in FIFO
NW	INT	Schreib-Index FIFO
NR	INT	Lese-Index FIFO

2.14. IP_CONTROL2

Type Funktionsbaustein

IN_OUT IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)

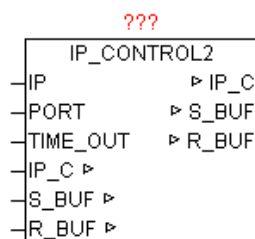
S_BUF: Datenstruktur 'NETWORK_BUFFER_SHORT'
(Sendedaten)

R_BUF: Datenstruktur 'NETWORK_BUFFER_SHORT'
(Empfangsdaten)

INPUT IP: DWORD (kodierte IP Adresse als Standardadresse)

PORT: WORD (Portnummer der IP-Adresse)

TIME_OUT: TIME (Überwachungszeit)



Der Baustein hat prinzipiell die gleiche Funktionalität wie IP_CONTROL. Jedoch sind S_BUF und R_BUF vom TYP 'NETWORK_BUFFER_SHORT' (Siehe allgemeine Beschreibung IP_CONTROL).

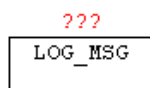
Es wird bei IP_CONTROL2 kein Blocken der Daten unterstützt. Die maximale Datengröße beim Senden und Empfangen ist abhängig von der Hardware-Plattform und bewegt sich im Bereich < 1500 Bytes. Dieser Baustein kann immer dann eingesetzt werden wenn kein Data-Stream Modus notwendig ist. Primärer Vorteil hierbei ist, das weniger Buffer-Speicher belegt wird, und keine Daten zwischen internen und externen

Daten Buffer kopiert werden muss. Somit ist der Baustein bezüglich Speicherverbrauch und Systembelastung sparsamer.

2.15. LOG_MSG

Type Funktionsbaustein:

IN_OUT LOG_CL: Datenstruktur 'LOG_CONTROL' (LOG-Daten)



Mit LOG_MSG können Meldungen (STRINGS) in einen RING-BUFFER abgelegt werden. Die Meldungen können mit Zusatzparameter versehen werden wie Frontcolor und Backcolor für die Ausgabe auf TELNET sowie ein Eintragsfilter durch Vorgabe eines Nachrichten-Level. Ist der Level der Nachricht grösser als der Vorgabe-Log-Level, so wird diese Nachricht verworfen. Weiters kann mit Enable auch das Logging allgemein deaktiviert werden. Somit ist es kein Problem viele Meldungen pro SPS-Zyklus zu archivieren. Der Nachrichtenbuffer kann z.B. mit dem Baustein TELNET_LOG auf einen TELNET-CLIENT ausgegeben werden. Details zur Schnittstelle sind in der nachfolgenden Tabelle ersichtlich.

Wenn aus verschiedenen Bausteininstanzen Meldungen in den selben LOG_BUFFER eingetragen werden sollen, so muss die „LOG_CL“ Datenstruktur GLOBAL angelegt werden.

LOG_CONTROL Datenstruktur:

(Nur die blaue Elemente dürfen vom Anwender verändert werden !)

Name	Typ	Eigenschaften
NEW_MSG	STRING(255)	Neue Meldung - Text
NEW_MSG_OPTION	DWORD	Neue Meldung – Option BYTE 3: Reserve BYTE 2: Level BYTE 1: Backcolor BYTE 0: Frontcolor
LEVEL	BYTE	Vorgegebener Log-Level
SIZE	INT	Größe des Array (maximaler Index)
RESET	BOOL	Rücksetzen / Löschen der Einträge
ENABLE	BOOL	Freigabe zum Eintragen von Meldungen

PRINTF	ARRAY[1.11] OF STRING(255)	Parameterdaten für PRINT_SF Baustein
MSG	ARRAY[0.N] OF STRING(255)	Array für Meldungen - Text
MSG_OPTION	ARRAY[0.N] OF DWORD	Array für Meldungen – Option BYTE 3: Reserve BYTE 2: Level BYTE 1: Backcolor BYTE 0: Frontcolor
UPDATE_COUNT	UINT	Update-Zähler (wird mit jeder neuen Nachricht erhöht)
IDX	INT	Aktueller Ausgabe-Index
RING_MODE	BOOL	BUFFER Überlauf / Ringmode aktiviert

2.16. MB_CLIENT (OPEN-MODBUS)

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER_SHORT' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER_SHORT' (Empfangsdaten) DATA: Datenstruktur 'MB_DATA' (MODBUS-Register)
INPUT	DATA_SIZE: INT (Anzahl der Datenwörter in Struktur DATA) ENABLE: BOOL (Freigabe) UDP: BOOL (Vorwahl TCP / UDP, TRUE = UDP) FC: INT (Funktionsnummer) UNIT_ID: BYTE (Geräte-Identifikationsnummer) R_ADDR: INT (Lesebefehl: MODBUS Datenpunkt Adresse) R_POINTS: INT (Lesebefehl: MODBUS Anzahl der Datenpunkte) R_DATA_ADR: INT (Lesebefehl: DATA Datenpunkt Adresse) R_DATA_BITPOS: INT (Lesebefehl: DATA Datenpunkt Bitposition) W_ADDR: INT (Lesebefehl: MODBUS Datenpunkt Adresse) W_POINTS: INT (Lesebefehl: MODBUS Anzahl der Datenpunkte)

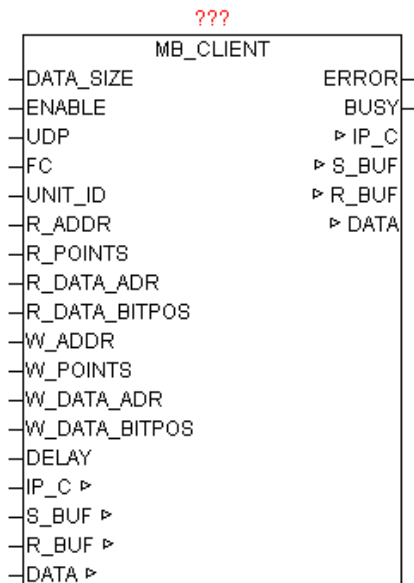
W_DATA_ADR: INT (Lesebefehl: DATA Datenpunkt Adresse)

W_DATA_BITPOS: INT (Lesebefehl: DATA Datenpunkt Bitposition)

DELAY: TIME (Wiederholungszeit)

OUTPUT ERROR: DWORD (Fehlercode)

BUSY: BOOL (Baustein ist aktiv)



Der Baustein ermöglicht den Zugriff auf Ethernet-Geräte die MODBUS-TCP bzw. MODBUS-UDP unterstützten, bzw. MODBUS RS232/485 Geräte die über Ethernet-Modbus-Gateway angebunden sind. Es werden Kommandos aus den Klassen 0,1,2 unterstützt. Die Parameter IP_C, S_BUF, R_BUF bilden hierbei die Schnittstelle zum IP_CONTROL Baustein der hier zur Abwicklung bzw. Koordinierung benutzt wird. Die gewünschte IP-Adresse und Port-Nummer (Standard für MODBUS ist 502) muss am IP_CONTROL zentral angegeben werden. Die DATA-Struktur ist als WORD-Array ausgeführt und beinhaltet die MODBUS-Daten für Lesen und Schreiben. Mittels DATA_SIZE wird die Größe des WORD_ARRAY angegeben. Durch ENABLE wird der Baustein freigegeben, und bei Wegnahme der Freigabe wird eine eventuell aktive Abfrage noch beendet. Bei Geräten die MODBUS-UDP unterstützen kann mittels UDP=TRUE dieser Modus aktiviert werden. Der Parameter UNIT_ID muss nur bei Nutzung eines Ethernet-Modbus-Gateway angegeben werden. Die gewünschte Funktion wird mittels FC angegeben (Siehe Funktionscode-Tabelle). Je nach Funktion müssen die R_xxx und W_xxx Parameter mit Daten versorgt werden. Durch Angabe DELAY kann die Wiederholungszeit vorgegeben werden. Wird keine Zeit angegeben so wird versucht so oft wie möglich das Kommando auszuführen. Durch die integrierte Zugriffsverwaltung ist automatisch sichergestellt, dass andere Baustein-Instanzen auch an die Reihe kommen. Eine negative Befehlsausführung wird mittels ERROR

gemeldet (Siehe ERROR-Tabelle). Wenn der Baustein aktiv eine Abfrage durchführt, dann wird während dieser Zeit BUSY=TRUE ausgegeben.

Unterstützte Funktionscodes und verwendete Parameter:

Funktionscode	Bit Zugriff	16 Bit Zugriff (Register)	Gruppe	Funktion Beschreibung	R_ADDR	R_POINTS	R_DATA_ADR	R_DATA_BITPOS	W_ADDR	W_POINTS	W_DATA_ADR	W_DATA_BITPOS
1	x		Coils	Read Coils	x	x	x	x				
2	x		Input Discrete	Read Discrete Inputs	x	x	x	x				
3		x	Holding Register	Read Holding Registers	x	x	x					
4		x	Input Register	Read Input Register	x		x					
5	x		Coils	Write Single Coil					x		x	x
6		x	Holding Register	Write Single Register					x		x	
15	x		Coils	Write Multiple Coils					x	x	x	x
16		x	Holding Register	Write Multiple Register					x	x	x	
22		x	Holding Register	Mask Write Register					x		x	
23		x	Holding Register	Read/Write Multiple Register	x	x	x		x	x	x	

ERROR:

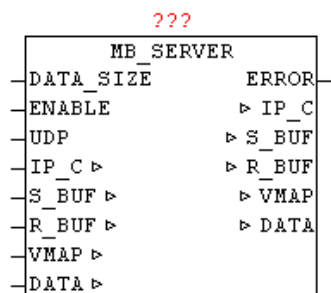
Wert				Ursprung	Beschreibung
B3	B2	B1	B0		
nn	nn	nn	xx	IP_CONTROL	Fehler vom Baustein IP_CONTROL
xx	xx	xx	00	MB_CLIENT	No Error
xx	xx	xx	01	MB_CLIENT	ILLEGAL FUNCTION: The function code received in the query is not an allowable action for the server (or slave). This may be because the function code is only applicable to newer devices, and was not implemented in the unit selected. It could also indicate that the server (or slave) is in the wrong state to process a request of this type, for example because it

					is unconfigured and is being asked to return register values.
xx	xx	xx	02	MB_CLIENT	ILLEGAL DATA ADDRESS: The data address received in the query is not an allowable address for the server (or slave). More specifically, the combination of reference number and transfer length is invalid. For a controller with 100 registers, the PDU addresses the first register as 0, and the last one as 99. If a request is submitted with a starting register address of 96 and a quantity of registers of 4, then this request will successfully operate (address-wise at least) on registers 96, 97, 98, 99. If a request is submitted with a starting register address of 96 and a quantity of registers of 5, then this request will fail with Exception Code 0x02 "Illegal Data Address" since it attempts to operate on registers 96, 97, 98, 99 and 100, and there is no register with address 100.
xx	xx	xx	03	MB_CLIENT	ILLEGAL DATA VALUE: A value contained in the query data field is not an allowable value for server (or slave). This indicates a fault in the structure of the remainder of a complex request, such as that the implied length is incorrect. It specifically does NOT mean that a data item submitted for storage in a register has a value outside the expectation of the application program, since the MODBUS protocol is unaware of the significance of any particular value of any particular register.
xx	xx	xx	04	MB_CLIENT	SLAVE DEVICE FAILURE: An unrecoverable error occurred while the server (or slave) was attempting to perform the requested action.
xx	xx	xx	05	MB_CLIENT	ACKNOWLEDGE: Specialized use in conjunction with programming commands. The server (or slave) has accepted the request and is processing it, but a long duration of time will be required to do so. This response is returned to prevent a timeout error from occurring in the client (or master). The client (or master) can next issue a Poll Program Complete message to determine if processing is completed.
xx	xx	xx	06	MB_CLIENT	SLAVE DEVICE BUSY: Specialized use in conjunction with programming commands. The server (or slave) is engaged in processing a long-duration program command. The client (or master) should retransmit the message later when the server (or slave) is free.
xx	xx	xx	8	MB_CLIENT	MEMORY PARITY ERROR: Specialized use in conjunction with function codes 20 and 21 and reference type 6, to indicate that the extended file area failed to pass a consistency check. The server (or slave) attempted to read record file, but detected a parity error in the memory. The client (or master) can retry the request, but service may be required on the server (or slave) device.
xx	xx	xx	0A	MB_CLIENT	GATEWAY PATH UNAVAILABLE: Specialized use in conjunction with gateways, indicates that the gateway was unable to allocate an internal communication path

					from the input port to the output port for processing the request. Usually means that the gateway is misconfigured or overloaded.
xx	xx	xx	0B	MB_CLIENT	GATEWAY TARGET DEVICE FAILED TO RESPOND: Specialized use in conjunction with gateways, indicates that no response was obtained from the target device. Usually means that the device is not present on the network.

2.17. MB_SERVER (OPEN-MODBUS)

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER_SHORT' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER_SHORT' (Empfangsdaten) VMAP: Datenstruktur 'MB_VMAP' (Virtuelle Adresstabellen) DATA: Datenstruktur 'MB_DATA' (MODBUS-Register)
INPUT	DATA_SIZE: INT (Anzahl der Datenwörter in Struktur DATA) ENABLE: BOOL (Freigabe) UDP: BOOL (Vorwahl TCP / UDP, TRUE = UDP)
OUTPUT	ERROR: DWORD (Fehlercode)



Der Baustein ermöglicht den Zugriff von Extern auf lokale MODBUS-Datentabellen über Ethernet. Es werden Kommandos der Klassen 0,1,2 unterstützt. Die Parameter IP_C, S_BUF, R_BUF bilden hierbei die Schnittstelle zum IP_CONTROL Baustein der hier zur Abwicklung bzw. Koordinierung benutzt wird. Die gewünschte Port-Nummer (Standard für MODBUS ist 502) muss am IP_CONTROL zentral angegeben werden. Die IP-Adresse wird am IP_CONTROL nicht benötigt, da dieser hier im Modus PASSIV agiert. Die DATA-Struktur ist als WORD-Array ausgeführt und

beinhaltet die MODBUS-Daten. Mittels DATA_SIZE wird die Größe von DATA angegeben. Durch ENABLE wird der Baustein freigegeben, und bei Wegnahme der Freigabe wird eine eventuell aktive Abfrage noch beendet. Bei Geräten die MODBUS-UDP unterstützen kann mittels UDP=TRUE dieser Modus aktiviert werden. Eine negative Befehlsausführung wird mittels ERROR gemeldet (Siehe ERROR-Tabelle).

Mittels Einträgen in der Datenstruktur VMAP können virtuelle Datenbereiche angelegt werden, und der Zugriff für bestimmte Funktionscodes und Datenbereiche parametrisiert werden. Somit ist es sehr einfach möglich virtuelle Adressbereiche in einen zusammenhängenden Datenblock (DATA) abzubilden, oder Datenbereiche mit einen Schreibschutz zu versehen, oder Bereiche die mit Ausgangsperipherie gekoppelt sind mit einer Watchdog zu versehen.

Die Handhabung der VMAP-Daten ist beim Baustein MB_VMAP näher beschrieben.

ERROR:

Wert				Ursprung	Beschreibung
B3	B2	B1	B0		
nn	nn	nn	xx	IP_CONTROL	Fehler vom Baustein IP_CONTROL
xx	xx	xx	00	MB_SERVER	NO ERROR:
xx	xx	xx	01	MB_SERVER	ILLEGAL FUNCTION:
xx	xx	xx	02	MB_SERVER	ILLEGAL DATA ADDRESS:
xx	xx	xx	03	MB_SERVER	ILLEGAL DATA VALUE:

Unterstützte Funktionscodes und verwendete Parameter:

Funktionscode	Bit Zugriff	16 Bit Zugriff (Register)	Gruppe	Funktion Beschreibung

1	x		Coils	Read Coils
2	x		Input Discrete	Read Discrete Inputs
3		x	Holding Register	Read Holding Registers
4		x	Input Register	Read Input Register
5	x		Coils	Write Single Coil
6		x	Holding Register	Write Single Register
15	x		Coils	Write Multiple Coils
16		x	Holding Register	Write Multiple Register
22		x	Holding Register	Mask Write Register
23		x	Holding Register	Read/Write Multiple Register

2.18. MB_VMAP

Type Funktionsbaustein:

IN_OUT VMAP : Datenstruktur 'VMAP' (VIRTUAL_MAP Daten)

INPUT FC: INT (Funktionsnummer)

V_ADR: INT (Virtuelle Adressbereich: Startadresse)

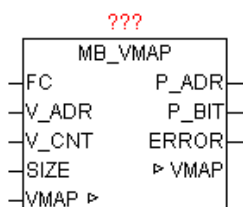
V_CNT: INT (Virtuelle Adressbereich: Anzahl der Datenpunkte)

SIZE: INT (Anzahl der Datenwörter in Struktur DATA)

OUTPUT: P_ADR: INT (Realer Adressbereich: Startadresse)

P_BIT: INT (Realer Adressbereich: Bitposition)

ERROR: DWORD (Fehlercode)



Der Baustein ermöglicht die Umrechnung von Virtuellen Adressen auf einen Realen Adressbereich im der MODBUS-DATA Struktur. Dazu werden

im VMAP-Datenarray Virtuelle Adressbereiche definiert. Wird der Baustein aufgerufen und dabei festgestellt das in den VMAP-Daten nichts eingetragen ist, wird automatisch ein Block angelegt, der den vollen Zugriff auf die ganzen MODBUS-Daten ermöglicht. In jedem Adressblock wird ein auch Watchdog-Timer verwaltet, der bei jedem Zugriff über diesem Block den Timer auf Null setzt. Somit kann einfach durch vergleichen des TIME_OUT Wertes mit einem Abschaltwert auf Kommunikationsstörungen (keine Aktualisierung) reagiert werden.

Mittels dem Parameter FC wird erkannt um welchen Funktionscode es sich handelt, und ob dabei Register (16Bit) oder einzelne Bits behandelt werden müssen. Die Bitnummer entspricht dem Funktionscode. Das heißt das z.B. BIT5=1 in FC den Funktionscode 5 (Write Single Coil) aktiviert. Durch V_ADR wird die virtuelle Startadresse angegeben (Bei 16Bit Befehlen ist dies eine Register-Adresse und bei Bit Befehlen eine absolute Bitnummer innerhalb eines definierten Blocks. Der Parameter V_CNT definiert die Anzahl der Datenpunkte (Einheit 16 Bit oder Bit je nach Funktionscode). Die Gesamtgröße des MODBUS_ARRAY wird über SIZE (Anzahl WORDS) vorgegeben. Anhand dieser Parameter durchsucht der Baustein die VMAP Datentabelle nach einen passenden Datenblock, und gibt vom ersten korrekten Datenblock die P_ADR als Ergebnis zurück. Der Wert entspricht den realen Index für MODBUS_DATA Array. Bei einem Funktionscode mit Bit Zugriff wird noch zusätzlich die Bit-Position innerhalb P_ADR mit ausgegeben. Ein möglicherweise auftretender Fehler bei der Auswertung wird beim Parameter „Error“ gemeldet (Siehe Error-Tabelle). Der Watchdog-Timer wird bei jeden Zugriff mit einem Funktionscode aus der Gruppe der schreibenden Kommandos zurückgestellt.

Wird keine Sonderbehandlung gewünscht so sind in VMAP keinerlei Einstellungen notwendig, und das MODBUS_ARRAY wird dann 1:1 beim Zugriff abgebildet.

ERROR:

Wert	Beschreibung
0	Keine Fehler
1	Ungültiger Funktionscode
2	Ungültige Datenadresse

! Hinweis zur Sonderbehandlung des Funktionscode 23 !

Der Modbus-Funktionscode 23 ist ein kombinierter Befehl, weil er aus zwei Aktionen besteht. Zuerst werden Register geschrieben, und danach

Register gelesen. Wird festgestellt das Schreib oder Lese-Parameter nicht zulässig sind, so wird keine der beiden Aktionen durchgeführt.

Damit über VMAP auch zwischen dem Lesen und Schreiben unterschieden werden kann, wird der Lesebefehl in VMAP bei FC 23 als BIT23 (Read/Write Multiple Register) , und der Schreibbefehl hingegen über BIT16 (Write Multiple Register) geprüft.

Die Datenstruktur MB_VMAP enthält folgende Felder:

Datenfeld	Datentyp	Beschreibung
FC	DWORD	Funktionscode: Freigabe-Bitmaske
V_ADR	INT	Virtueller Adressbereich: Startadresse
V_SIZE	INT	Virtueller Adressbereich: Anzahl Word
P_ADR	INT	Realer Adressbereich: Startadresse
TIME_OUT	TIME	Watchdog

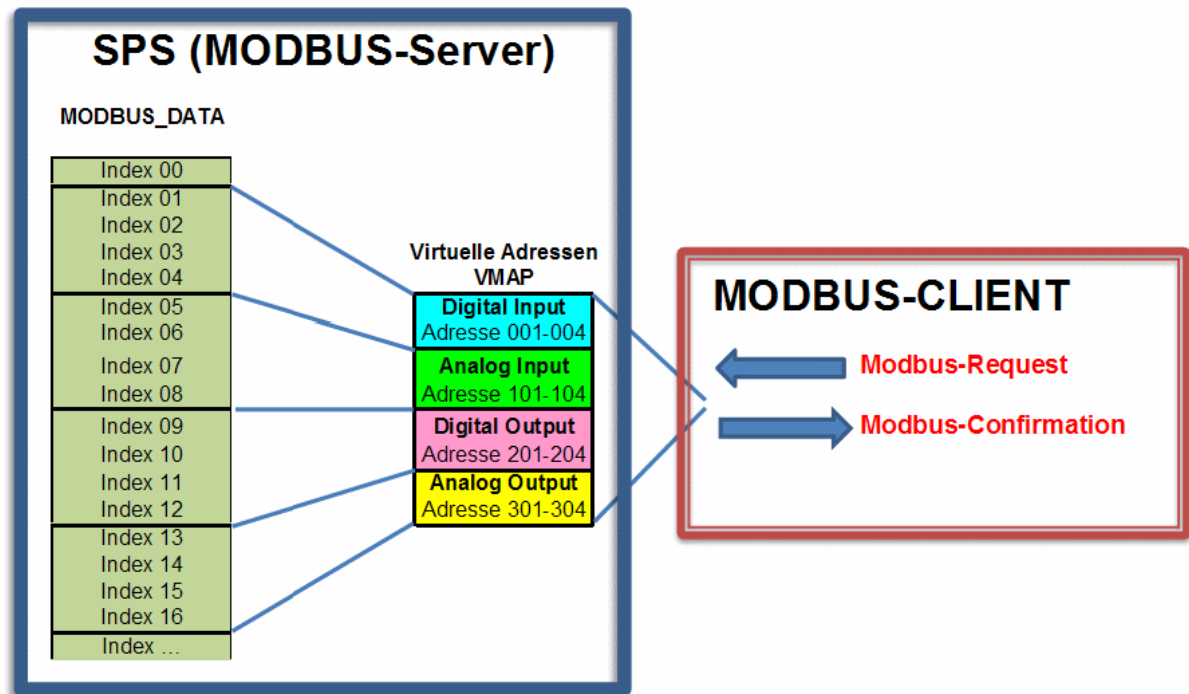
Beispielkonfiguration

```
(* Virtueller Block 1 *)
VMAP[1].FC := DWORD#2#00000000_10000000_00000000_00011100); (FC 2,3,4,23)
VMAP[1].V_ADR := 1; (* Virtueller Adressbereich: Startadresse *)
VMAP[1].V_SIZE := 4; (* Virtueller Adressbereich: Anzahl der WORD *)
VMAP[1].P_ADR := 1; (* Realer Adressbereich: Startadresse *)
(* Virtueller Block 2 *)
VMAP[2].FC := DWORD#2#00000000_10000000_00000000_00011000); (FC 3,4,23)
VMAP[2].V_ADR := 101; (* Virtueller Adressbereich: Startadresse *)
VMAP[2].V_SIZE := 4; (* Virtueller Adressbereich: Anzahl der WORD *)
VMAP[2].P_ADR := 5; (* Realer Adressbereich: Startadresse *)
(* Virtueller Block 3 *)
VMAP[3].FC := DWORD#2#00000000_11000001_10000000_01111010); (FC1,3-6,15-16,23)
VMAP[3].V_ADR := 201; (* Virtueller Adressbereich: Startadresse *)
VMAP[3].V_SIZE := 4; (* Virtueller Adressbereich: Anzahl der WORD *)
VMAP[3].P_ADR := 9; (* Realer Adressbereich: Startadresse *)
(* Virtueller Block 4 *)
VMAP[4].FC := DWORD#2#00000000_11000001_00000000_01011000); (FC 3,4,6,16,23)
VMAP[4].V_ADR := 301; (* Virtueller Adressbereich: Startadresse *)
```

```

VMAP[4].V_SIZE := 4; (* Virtueller Adressbereich: Anzahl der WORD *)
VMAP[4].P_ADR := 12;      (* Realer Adressbereich: Startadresse *)

```



Die Konfiguration ergibt folgende Zugriffs-Matrix:

Funktion Beschreibung	Funktionscode	Bit Zugriff	16 Bit Zugriff (Register)	Lesen / Schreiben		Digital Input	Analog Input	Digital Output	Analog Output
Read Coils	1	x		Lesen				x	
Read Discrete Inputs	2	x		Lesen	x				
Read Holding Registers	3		x	Lesen	x	x	x	x	
Read Input Register	4		x	Lesen	x	x	x	x	
Write Single Coil	5	x		Schreiben				x	
Write Single Register	6		x	Schreiben				x	x
Write Multiple Coils	15	x		Schreiben				x	

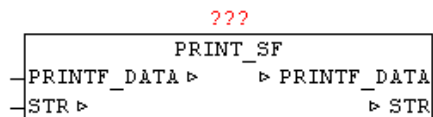
Write Multiple Register	16		x	Schreiben			x	x
Mask Write Register	22		x	Schreiben				
Read/Write Multiple Register	23		x	Lesen / Schreiben	x	x	x	x

2.19. PRINT_SF

Type Funktionsbaustein:

IN_OUT PRINTF_DATA : ARRAY[1..11] OF STRING(255) (Parameterdaten)

STR: STRING(255) (Ergebnisstring)



Mit PRINT_SF kann ein STRING mit Teilstrings dynamisch ergänzt werden. Die Position der einzufügenden Teilstrings wird mittels '~' Tilde Zeichen angegeben und die nachfolgenden Nummer definiert die Parameternummer. '~1' bis '~9' werden somit automatisch verarbeitet. Wird beim einfügen des Teilstrings das Limit von 255 Zeichen erreicht so wird anstatt des Teilstring nur mit '..' eingefügt.

VAR

LITER : REAL := 545.4;

FUELLZEIT : INT := 25;

NAME : STRING := 'Tankinhalt';

PARA : ARRAY[1..11] OF STRING(255);

PS : PRINT_SF;

END_VAR

PARA[1] := REAL_TO_STRING(Liter); (* Parameter 1: in String wandeln *)

PARA[2] := INT_TO_STRING(Fuellzeit); (* Parameter 2: in String wandeln *)

PARA[3] := NAME; (* Parameter 3: *)

PS.STR := '~3: ~1 Liter, Füllzeit: ~2 Min.'; (* Textausgabe-Maske *)

PS.PRINTF_DATA := PARA; (* Parameter Datenstruktur übergeben *)

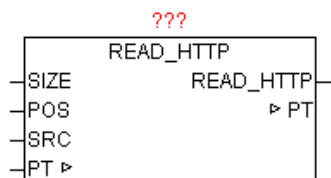
PS(); (* Baustein ausführen *)

Der String PS.STR hat danach folgenden Inhalt

'Tankinhalt: 545.4 Liter, Füllzeit: 25 Min.'

2.20. READ_HTTP

Type	Funktionsbaustein:	
INPUT	SIZE : UINT	(Größe des Puffers)
	POS : INT	(Position ab der gesucht wird)
	SRC : STRING	(Suchstring)
IN_OUT	PT : POINTER	(Adresse des Puffers)
OUTPUT	VALUE	(Parameter der Header-Information)



Nach einem erfolgreichem HTTP-GET Request sind immer ein HTTP-Header (Message Header) und ein Nachrichtenkörper (Message Body) im Buffer vorhanden. Im HTTP-Header sind diverse Informationen zur angefragten HTTP-Seite abgelegt. Der nachfolgende Nachrichtenkörper beinhaltet die eigentlichen angefragten Daten. Mittels READ_HTTP können die HTTP-Header-Informationen ausgewertet werden. Der Baustein durchsucht ein beliebiges Array of Byte auf den Inhalt einer Zeichenkette und wertet danach den nachfolgenden Parameter aus, und liefert diesen String als Ergebnis zurück. Mit POS kann an beliebiger Position mit der Suche begonnen werden. Das erste Element im Array hat die Positionsnummer 1.

Beispiel einer HTTP-Response (Header-Informationen):

HTTP/1.0 200 OK<CR><LF>

Content-Length: 2165<CR><LF>

Content-Type: text/html<CR><LF>

Date: Mon, 15 Sep 2008 16:59:08 GMT<CR><LF>

Last-Modified: Wed, 18 Jun 2008 12:35:52 GMT<CR><LF>

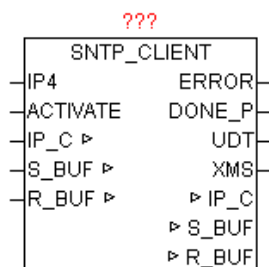
Mime-Version: 1.0<CR><LF><CR><LF>

Wenn SRC keinen Suchbegriff beinhaltet, wird automatisch die HTTP Version und der HTTP Statuscode im Buffer besucht und ausgewertet. Als

Ergebnis wird nach obigen Beispiel „1.0 200 OK“ zurückgegeben. Ist in SRC ein Suchbegriff enthalten wird diese Header-Information im Buffer gesucht und der Wert als String zurückgegeben z.B. 'Content-Length' = „2165“.

2.21. SNTP_CLIENT

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
INPUT	IP4: DWORD (IP-Adresse des SNTP-Servers) ACTIVATE: BOOL (Startet die Abfrage)
OUTPUT	ERROR: DWORD (Fehlercode) DONE_P: BOOL (positiver Flanke Fertig ohne Fehler) UDT: DT (Datums und Zeit Ausgang als Weltzeit) XMS: INT (Millisekunden der Weltzeit UDT)

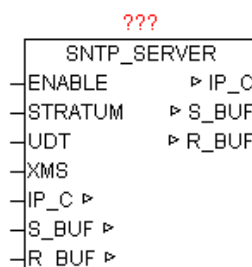


Der SNTP_CLIENT dient zum synchronisieren der lokalen Uhrzeit mit einem SNTP-Server. Dazu wird das Simple-Network-Time-Protokoll verwendet, das speziell entwickelt wurde, um eine zuverlässige Zeitangabe über Netzwerke mit variabler Paketlaufzeit zu ermöglichen. Das SNTP ist Datentechnisch völlig identisch mit NTP, das heißt hier bestehen keinerlei Unterschiede. Somit können sämtliche gekannte SNTP und NTP Server ob im lokalen Netzwerk als auch über das Internet genutzt werden. Bei IP4 wird die IP-Adresse eines SNTP/NTP Server angegeben. Eine positive Flanke bei ACTIVATE startet die Abfrage. Die vergangene Zeit zwischen Senden und dem Empfang der Zeit wird gemessen und daraus wird eine Zeitkorrektur errechnet. Danach wird die empfangene Zeit um diesen Wert korrigiert. Bei erfolgreicher Beendigung gibt DONE_P eine positiven Flanke aus, und die aktuelle Zeit wird bei UDT ausgegeben. Weiters werden auf XMS noch die zugehörigen Sekundenbruchteile als Millisekunden ausgegeben. Die Werte von UDT und XMS sind nur bei DONE_P = TRUE gültig, da es sich um einen statischen Uhrzeitwert

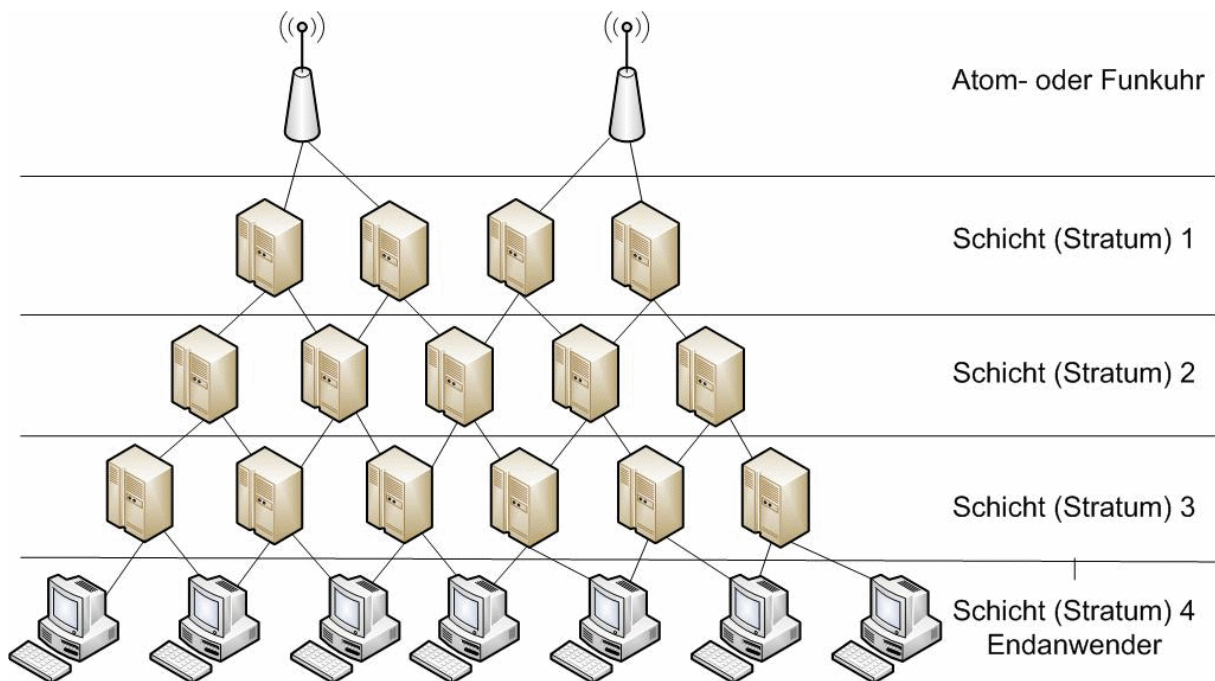
handelt, und dienen nur zum Impuls gesteuerten Uhrzeitstellen. ERROR liefert im Fehlerfall die genaue Ursache (Siehe Baustein IP_CONTROL).

2.22. SNTP_SERVER

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
INPUT	ENABLE: BOOL (Startet die SNTP-Server) STRATUM: BYTE (Angabe der hierarchische Ebene bzw. Genauigkeit) UDT: DT (Datums und Zeit Eingang als Weltzeit) XMS: INT (Millisekunden der Weltzeit UDT)



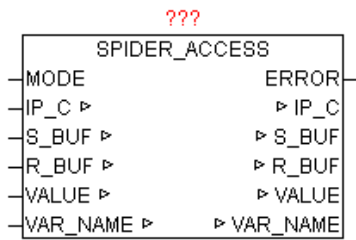
Der Baustein stellt die Funktionalität eines SNTP (NTP) Servers zu Verfügung. Bei ENABLE = TRUE meldet sich der Baustein bei IP_CONTROL an und wartet auf Freigabe der Ressource, sollte diese von anderen Teilnehmern momentan noch belegt sein. Danach wartet der Baustein auf Anfragen von anderen SNTP-Clients und beantwortet diese mit der aktuellen Uhrzeit von UDT und XMS. Solange ENABLE = TRUE ist, wird der Zugriff auf diese Ethernet-Ressource dauerhaft für andere Teilnehmer gesperrt (Exklusiv-Zugriff - wegen Passiv UDP Modus). SNTP nutzt ein hierarchisches System verschiedener Strata. Als Stratum 0 bezeichnet man das exakte Zeitnormal. Die unmittelbar mit ihr gekoppelten Systeme, beispielsweise NTP, GPS oder DCF77 Zeitsignale heißen Stratum 1. Jede weitere abhängige Einheit verursacht einen zusätzlichen Zeitversatz von 10-100ms und erhält bei der Bezeichnung eine höhere Nummer (Stratum 2, Stratum 3 bis 255). Wird kein STRATUM am Baustein angegeben wird STRATUM=1 als Standard verwendet.



Wenn ein SNTP-Client selber eine Uhrzeit mit höheren Stratum als eine SNTP-Server hat, wird die Uhrzeit von diesen mitunter abgelehnt, da diese ungenauer ist, als die eigene Referenz. Darum ist es wichtig ein logisch richtiges STRATUM vorzugeben. Der Baustein SNTP_CLIENT ignoriert jedoch absichtlich das STRATUM und synchronisiert sich auf jeden Fall mit dem SNTP-Server, da ziemlich jeder SNTP-Server eine genauere Uhrzeit besitzt, als eine SPS.

2.23. SPIDER_ACCESS

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten) VALUE: STRING (Wert der Variable) NAME: STRING(40) (Variablenname)
INPUT	MODE: BYTE (Betriebsmodus: 1= lesen / 2=schreiben)
ERROR	ERROR: DWORD (Fehlercode)

**ERROR:**

Wert	Eigenschaften
1	Beim Schreiben von Variablenwerte ist ein Fehler aufgetreten
> 1	Die genaue Bedeutung von ERROR ist beim Baustein HTTP_GET nachzulesen

Mit SPIDER_ACCESS können von Steuerungen die Visualisierungen über Webserver auf Basis „SpiderControl“ von Fa. IniNet Solution GmbH integriert haben, Variablen gelesen und geschrieben werden.

Für folgende Steuerungen gibt es diese Webserver Integration:

Simatic S7 200/300/400
 SAIA-Burgess PCD
 Wago (750-841)
 Beckhoff (CX Reihe)
 Phoenix Contact (ILC Reihe)
 Selectron
 Berthel
 Tbox
 Beck IPC

Im SPS-Programm der Zielsteuerung müssen die gewünschten Variablen für den Webzugriff freigegeben sein. Da die Kommunikation mittels HTTP (Port 80) durchgeführt wird, kann auch über Firewalls hinweg der Datenaustausch problemlos erfolgen. Es können Globale als auch Instanz-Variablen verarbeitet werden.

Format der Variablen:

Bei einer Globalen Variablen muss nur der normale Variablenname angegeben werden. Eine Instanz-Variable muss folgend angegeben werden. „instanzname.Variablenname“

Modus: Lesen

Wird der Parameter MODE auf „1“ gesetzt und bei „NAME“ der Variablenname angegeben, so wird zyklisch eine HTTP Anfrage an den

Webserver (SPS) durchgeführt, und das gemeldete Ergebnis bei „VALUE“ als STRING ausgegeben.

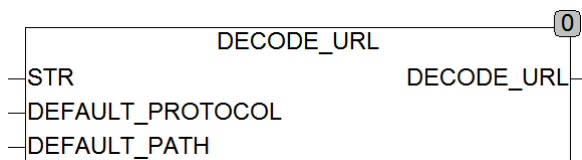
Modus: Schreiben

Wird der Parameter MODE auf „2“ gesetzt und bei „VALUE“ der Variablenwert und bei „NAME“ der Variablenname als STRING angegeben, so wird zyklisch eine HTTP Anfrage an den Webserver (SPS) durchgeführt

Der Modus bzw. der Variablenname kann im zyklischen Betrieb jederzeit geändert werden. Sollten mehrere Variablen verarbeitet werden müssen, so müssen lediglich dementsprechend viele Bausteininstanzen aufgerufen werden.

2.24. STRING_TO_URL

Type	Funktion : URL
Input	STR : STRING (Unified Resource Locator) DEFAULT_PROTOCOL : STRING (Ersatzprotokoll) DEFAULT_PATH : STRING (Ersatzpfad)
Output	URL (URL)



STRING_TO_URL zerlegt eine URL (Uniform Resource Locator) in seine Bestandteile und speichert diese in dem Datentyp URL. Wird in STR kein Pfad oder kein Protokoll spezifiziert, so setzt die Funktion die fehlenden Werte Automatisch mit den spezifizierten Ersatzwerten.

Eine URL setzt sich wie folgt zusammen:

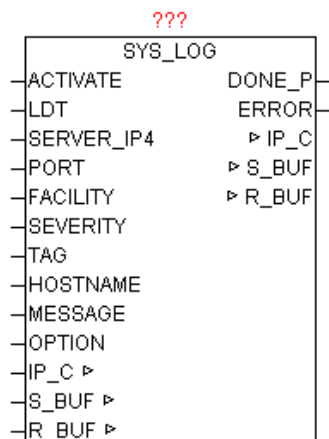
protokoll://user:passwort@domain:port/path?query#anker

Beispiel: <ftp://hugo:nono@oscat.de:1234/download/manual.html>

einige Bestandteile der URL sind optional wie zum Beispiel User, Passwort, Anker, Query ...

2.25. SYS_LOG

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten) S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten) R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
INPUT	ACTIVATE: BOOL (positive Flanke startet die Abfrage) LDT: DT (Lokalzeit) SERVER_IP4: DWORD (IP-Adresse des SYS-LOG Servers) PORT: WORD (Port-Nummer des SYS-LOG Servers) FACILITY: BYTE (spezifiziert den Dienst oder die Komponente) SEVERITY: BYTE (Klassifizierung des Schweregrades) TAG: STRING(32) (Prozessname , ID etc.) HOSTNAME: STRING (Name oder IP-Adresse des Senders) MESSAGE: STRING(255) (Nachrichtentext) OPTION: BYTE (diverse)
OUTPUT	DONE: BOOL (Abfrage ohne Fehler beendet) ERROR: DWORD (Fehlercode)



SYSLOG ist ein Standard zur Übermittlung von Meldungen in einem IP-Rechner-Netzwerk. Das Protokoll ist dabei sehr einfach aufgebaut – der Client sendet eine kurze Textnachricht an den SYSLOG-Empfänger. Der Empfänger wird auch als "syslog daemon" oder "syslog server" bezeichnet. Die Meldungen werden mittels UDP-Port 514 oder über TCP-Port 1468 gesendet und enthalten die Nachricht im Klartext. SYSLOG wird typischerweise für Computersystem-Management und Sicherheits-Überwachung benutzt. Damit ermöglicht es die leichte Integration von verschiedensten LOG-Quellen auf einen zentralen SYSLOG-Server. Die

Server-Software gibt es für alle Plattformen mitunter auch als Free / Shareware. Unix bzw. Linux-Systeme haben einen SYSLOG-SERVER schon integriert. Durch eine positive Flanke von ACTIVATE wird aus den Parametern LDT, FACILITY, SEVERITY, TAG, HOSTNAME, MESSAGE eine SYSLOG-Nachricht generiert und an die SERVER_IP4 Adresse gesendet. Mittels OPTION können noch diverse Eigenschaften gesteuert werden (Siehe Tabelle OPTION). Nach erfolgreichen Versenden wird DONE=TRUE, ansonsten wird bei ERROR die konkrete Fehlermeldung ausgegeben (Siehe ERROR von Baustein IP_CONTROL).

Eine Syslog-Message hat folgenden Aufbau

FACILITY, SEVERITY, TIMESTAMP, HOSTNAME, TAG, MESSAGE

Beispiel:

MAIL.ERR: Sep 10 08:31:10 149.100.100.02 PLANT2_PLC1 This is a test message generated by OSCAT SYSLOG

Folgende OPTION können verwendet werden

BIT	Funktion
0	FALSE = mit Facility, Severity-Code TRUE = ohne Facility, Severity-Code
1	FALSE = mit RFC Header TRUE = ohne RFC Header (nur die MESSAGE alleine wird versendet)
2	FALSE = mit CR, LF am Ende TRUE = ohne CR, LF am Ende
3	FALSE = UDP Modus TRUE = TCP Modus

Folgende Severity sind als Standard definiert:

Severity	Beschreibung
0	Emergency
1	Alert
2	Critical
3	Error
4	Warning

5	Notice
6	Informational
7	Debug

Folgende Facility sind als Standard definiert:

Facility	Beschreibung
00	Kernel message
01	user-level messages
02	mail system
03	system daemons
04	security/authorization messages
05	messages generated internally by syslogd
06	line printer subsystem
07	network news subsystem
08	UUCP subsystem
09	clock daemon
10	security/authorization messages
11	FTP daemon
12	NTP subsystem
13	log audit
14	log alert
15	clock daemon
16	local10
17	local11
18	local12
19	local13
20	local14
21	local15
22	local16

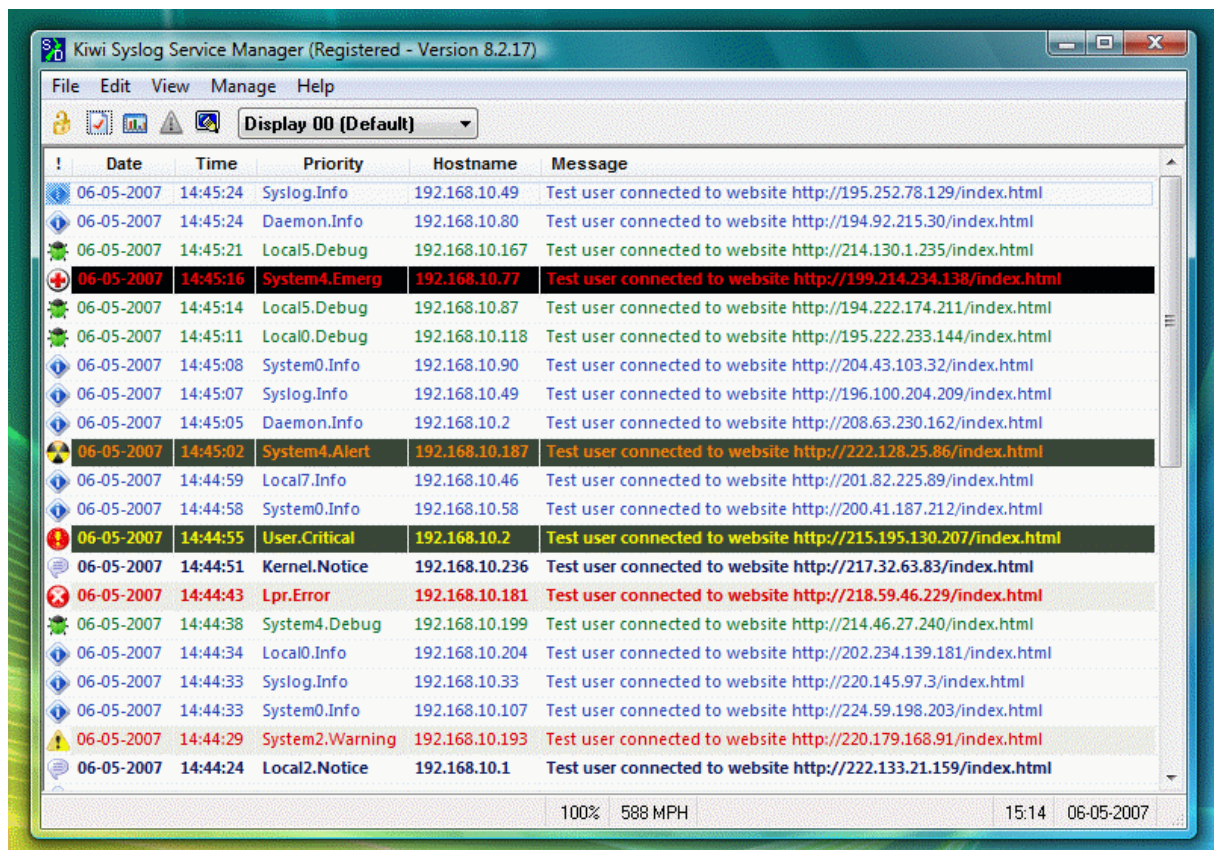
23

local17

Für allgemeine syslog-Nachrichten sind die Facility-Werte 16-23 vorgesehen (local0 bis local7). Es ist aber durchaus zulässig, auch die vordefinierten Werte 0 bis 15 für eigene Zwecke zu verwenden.

Mit Hilfe von Facility und Severity kann später auf den SYSLOG-Server (Datenbank) sehr leicht nach bestimmten Meldungen gefiltert werden, wie beispielsweise: "Erfasse alle Mailserver-Nachrichten vom Schweregrad error".

Beispiel (Screenshot) eines SYSLOG-Servers für Windows

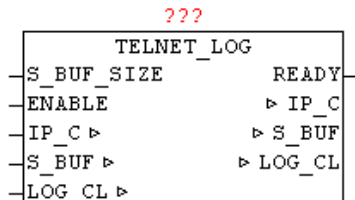


2.26. TELNET_LOG

Type Funktionsbaustein:

IN_OUT IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)

	S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten)
	LOG_CL: Datenstruktur 'LOG_CONTROL' (LOG-Daten)
INPUT	S_BUF_SIZE: UINT (Größe des S_BUF)
	ENABLE: BOOL (TELNET Server freigeben)
OUTPUT	READY: BOOL (TELNET Client hat Verbindung aufgebaut)



TELNET_LOG wird benutzt um alle Meldungen die sich im LOG_CONTROL Ring-buffer befinden über TELNET auszugeben. Durch „ENABLE“ kann der Baustein aktiviert werden. Sobald sich dann ein TELNET-Client über Port 23 verbindet wird dies über Parameter „READY“ angezeigt. Daraufhin werden automatisch alle Meldungen an TELNET ausgegeben. Sobald im weiteren Verlauf neue Meldungen in LOG_CONTROL auflaufen werden diese immer wieder automatisch auch ausgegeben. Bei einer erneuten Verbindung ab/aufbau werden wieder alle Meldungen erneut ausgegeben. Die meisten TELNET-Clients bieten auch die Möglichkeit den Datenstrom in eine Datei umzuleiten, um hier eine langfristige Datenarchivierung zu ermöglichen.

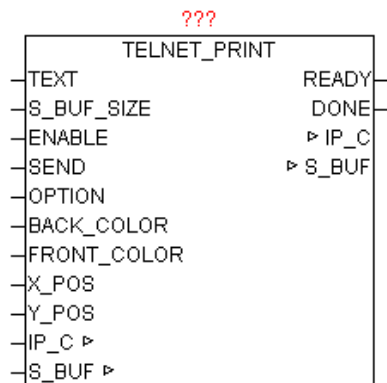
2.27. TELNET_PRINT

Type	Funktionsbaustein:
IN_OUT	IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)
	S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten)
INPUT	TEXT: STRING(255) (Ausgabe Text)
	S_BUF_SIZE: UINT ((Größe des S_BUF-Puffers)
	ENABLE: BOOL (Freigabe Kommunikation)
	SEND: BOOL (positive Flanke - Sendeanstoß)
	OPTION: BYTE (Sende-Optionen)
	BACK_COLOR: BYTE (Hintergrundfarbe)
	FRONT_COLOR: BYTE (Vordergrundfarbe)
	X_POS: BYTE (X-Koordinate der Schreibposition)

Y_POS: BYTE (Y-Koordinate der Schreibposition)

OUTPUT: READY: BOOL (Baustein bereit)

DONE: BOOL (positive Flanke - Senden beendet)



Der Baustein ermöglicht die einfache Ausgabe von Texten an eine TELNET-Konsole. Beim Parameter TEXT wird der gewünschte String übergeben. Um den Baustein für die Kommunikation freizuschalten muss ENABLE=1 gesetzt werden, damit erfolgt die Anmeldung beim IP_CONTROL. Mittels BACK_COLOR und FRONT_COLOR können die gewünschten Farben vorgegeben werden, vorausgesetzt die Funktion ist Parameter OPTION aktiviert. Die Parameter X_POS und Y_POS geben die gewünschte Koordinate der TEXT Ausgabe an. Wird bei X_POS und Y_POS der WERT „0“ angegeben, so ist die Textpositionierung inaktiv, und die Texte werden immer an der aktuellen Schreib-Cursor Position angehängt. Die Standard Telnet-Konsole erlaubt eine X_POS (Horizontal) von 1 bis 80 und eine Y_POS (Vertikal) von 1 bis 25. Das Verhalten hier kann wiederum mittels OPTION beeinflusst werden (Autowrap, Carriage-Return, Line-Fedd, Buf_Flush etc..). Wenn eine große Menge an Texten auf einmal ausgegeben werden muss, so kann eine Bufferung aktiviert werden, sodass die Daten erst geschrieben werden wenn entweder der Buffer voll ist (dies wird vom Baustein selbstständig veranlasst), oder dies durch den geänderten OPTION Parameter signalisiert wird. Durch SEND=1 werden die Daten in den Buffer geschrieben. Die Parameter dürfen erst wieder verändert werden wenn READY=1 ist, und mittels DONE wird die erfolgte Datenübernahme als positive Flanke angezeigt.

OPTION:

BIT	Funktion	Beschreibung
0	SCREEN_INIT	Nach dem Verbindungsaufbau mit der TELNET-Konsole wird der gesamte Bildschirm gelöscht. Wenn die OPTION COLOR aktiviert ist, wird der Bildschirm mit BACK_COLOR gelöscht.
1	AUTOWRAP	Bei AUTOWRAP=1 wird der Schreib-Cursor bei Erreichen

		des Zeilenendes automatisch auf eine nächste Zeile gesetzt. Wenn bei der Textausgabe die X,Y Positionen immer mit angegeben werden , ist es besser wenn AUTOWRAP=0 ist.
2	COLOR	Aktiviert die den Farbe-Modus , dabei werden BACK_COLOR und FRONT_COLOR bei der Ausgabe angewandt.
3	NEW_LINE	Bei NEW_LINE=1 wird automatisch am Ende des Textes ein Carriage Return und Line-Feed angehängt. So dass die nächste Textausgabe in einer neuen Zeile beginnt. Dies ist aber nur dann Sinnvoll, wenn keine X_POS und Y_POS vorgegeben werden.
4	RESERVE	
5	RESERVE	
6	RESERVE	
7	NO_BUF_FLUSH	Verhindert das die Daten im Buffer sofort gesendet werden. Nur wenn der Buffer komplett gefüllt ist, oder diese Option deaktiviert ist, werden die Daten versendet. Ermöglicht das schnelle Senden von vielen Texten im selben Zyklus

FRONT_COLOR:

Byte	Farbe	Byte	Farbe
0	Black	16	Flashing Black
1	Light Red	17	Flashing Light Red
2	Light Green	18	Flashing Light Green
3	Yellow	19	Flashing Yellow
4	Light Blue	20	Flashing Light Blue
5	Pink / Light Magenta	21	Flashing Pink / Light Magenta
6	Light Cyan	22	Flashing Light Cyan
7	White	23	Flashing White
8	Black	24	Flashing Black
9	Red	25	Flashing Red
10	Green	26	Flashing Green
11	Brown	27	Flashing Brown
12	Blue	28	Flashing Blue

13	Purple / Magenta	29	Purple / Magenta
14	Cyan	30	Flashing Cyan
15	Gray	31	Flashing Gray

BACK_COLOR:

Byte	Farbe
0	Black
1	Red
2	Green
3	Brown
4	Blue
5	Purple / Magenta
6	Cyan
7	Gray

The screenshot shows a HyperTerminal window titled "ddd - HyperTerminal". The menu bar includes "Datei", "Bearbeiten", "Ansicht", "Anrufen", and "Übertragung". The toolbar contains icons for file operations and terminal settings. The main display area shows the following text, each on a new line with a different background color:

```

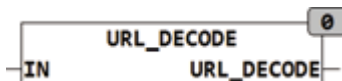
OSCAT Telnet_Print - Test: Color: 09
OSCAT Telnet_Print - Test: Color: 03
OSCAT Telnet_Print - Test: Color: 04
OSCAT Telnet_Print - Test: Color: 10
OSCAT Telnet_Print - Test: Color: 02
OSCAT Telnet_Print - Test: Color: 05
OSCAT Telnet_Print - Test: Color: 11
OSCAT Telnet_Print - Test: Color: 07
OSCAT Telnet_Print - Test: Color: 06

```

The status bar at the bottom displays "Verbunden 00:00:12", "ANSIW", "TCP/IP", "RF", "GROSS", "NUM", "Aufzeichnen", and "Druckerecho".

2.28. URL_DECODE

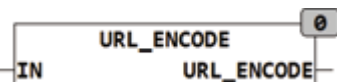
Type Funktion : STRING(255)
Input IN : STRING (Zeichenkette)
Output STRING(255) (Zeichenkette)



URL_DECODE wandelt die als %HH codierten Sonderzeichen in der Zeichenkette IN in den entsprechenden ASCII Code um. In einer URL Kodierung dürfen nur die Zeichen [A..Z, a..z, 0..9, -._~] vorkommen. Anderen Zeichen können mit einem % Zeichen gefolgt vom 2 Zeichen langen Hexadezimalcode des Zeichens dargestellt werden. Das reservierte Zeichen '#' wird dabei als '%23' kodiert.

2.29. URL_ENCODE

Type Funktion : STRING(255)
Input IN : STRING (Zeichenkette)
Output STRING(255) (Zeichenkette)



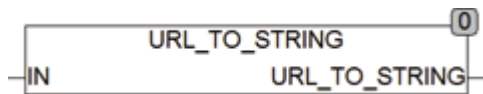
URL_ENCODE wandelt reservierte Zeichen in der Zeichenkette IN in die Zeichenkette '%HH' um. In einer URL Kodierung dürfen nur die Zeichen [A..Z, a..z, 0..9, -._~] vorkommen. Anderen Zeichen können mit einem % Zeichen gefolgt vom 2 Zeichen langen Hexadezimalcode des Zeichens dargestellt werden. Das reservierte Zeichen '#' wird dabei als '%23' kodiert.

2.30. URL_TO_STRING

Type Funktion : STRING(255)

Input IN : STRING (Unified Resource Locator)

Output URL (URL)



URL_TO_STRING erzeugt aus den in IN gespeicherten Daten vom Typ URL einen Unified Resource Locator als String zusammen.

Eine URL setzt sich wie folgt zusammen:

protokoll://user:passwort@domain:port/path?query#anker

Beispiel: <ftp://hugo:nono@oscat.de:1234/download/manual.html>

einige Bestandteile der URL sind optional wie zum Beispiel User, Passwort, Anker, Query ...

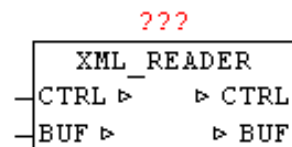
2.31. XML_READER

Type Funktionsbaustein:

IN_OUT CTRL : Datenstruktur 'XML_CONTROL'

(Steuer und Statusdaten)

BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)



Die Datenstruktur XML_CONTROL enthält folgende Felder:

Datenfeld	Datentyp	Beschreibung
COMMAND	WORD	Steuerbefehle (Binäre Belegung)
START_POS	UINT	(Buffer-Index des ersten Zeichen)
STOP_POS	UINT	(Buffer-Index des letzten Zeichen)
COUNT	UINT	Elementnummer
TYP	INT	Type-Code des aktuellen Element
LEVEL	UINT	Aktuelle Hierarchie / Ebene
PATH	STRING(255)	Hierarchie als TEXT (PFAD)

ELEMENT	STRING(255)	Aktuelles Element als TEXT
ATTRIBUTE	STRING(255)	Aktuelles Attribute als TEXT
VALUE	STRING(255)	Aktueller Value als TEXT
BLOCK1_START	UINT	Start-Position von Block 1
BLOCK1_STOP	UINT	Stopp-Position von Block 1
BLOCK2_START	UINT	Start-Position von Block 2
BLOCK2_STOP	UINT	Stopp-Position von Block 2

Mittels XML_READER ist es möglich so genannte 'Well-formed' XML-Dokumente zu parsen. Hierbei werden nicht wie bei Hochsprachen üblich, die ganze XML-Daten eingelesen und als Datenstruktur im Speicher abgelegt, sondern es wird eine sehr Ressourcen schonende Variante angewandt. Der XML_READER liest XML-Daten als sequentiellen Datenstrom aus dem Buffer und meldet die im COMMAND definierte Elemente-Typen automatisch zurück.

Bei XML wird strikt zwischen Groß- und Kleinschreibung unterschieden. Ein XML-gerechtes Dokument besteht aus Elementen, Attributen, ihren Wertzuweisungen, und dem Inhalt der Elemente, der aus Text oder aus untergeordneten Elementen bestehen kann, die ihrerseits wieder Attribute mit Wertzuweisungen und Inhalt haben können. Es gibt Elemente mit und ohne Attribute, Elemente, innerhalb deren viele andere Elemente vorkommen können, und solche, innerhalb deren nur Text vorkommen kann, und sogar leere Elemente, die keinen Inhalt haben können. Die Struktur, die aus diesen Bestandteilen und ihren Grundregeln entsteht, lässt sich als Baumstruktur begreifen. Elemente bestehen immer aus Tags und End-Tags. Attribute sind zusätzliche Informationen zu Elementen. Es sind auch Kommentar-Elemente erlaubt, jedoch dürfen sich diese beim XML_READER nicht zwischen Start- und End-Tags von Elementen befinden. Die möglichen DTD - Document Type Definition werden nur als DTD-Element gemeldet, jedoch nicht weiter vom XML_READER inhaltlich ausgewertet und angewandt. Mit einem CDATA-Abschnitt wird einem Parser mitgeteilt, dass kein Markup folgt, sondern normaler Text, der mittels Start-End-Block zurück gemeldet.

Vor dem ersten Aufruf des XML_READER müssen ein paar Parameter in der CTRL Datenstruktur initialisiert werden. Mittels CTRL.START_POS und CTRL.STOP_POS wird der Beginn und das Ende der XML-Daten im Buffer definiert. Mit CTRL.COMMAND kann einerseits ein Initialisierung (BIT15=TRUE) angestoßen werden, und mit Bit 0-14 kann definiert werden welche Element/Datentypen zurückgemeldet werden sollen. Dabei entsprechen die Typ-Codes der nachfolgenden Tabelle genau der Bit-Nummer die im CTRL.COMMAND dann jeweils auf TRUE gesetzt werden müssen.

Es wird immer versucht den Text von Element, Attribute, Value und Path in gesamter Länge an zugehörigen STRINGS zu übergeben. Bei STRINGS größer 255 Zeichen werden diese linksbündig abgeschnitten, jedoch mittels Block-Start und Block-End Parameter sehr wohl zurückgemeldet, so dass diese nachträglich trotzdem komplett ausgewertet werden können. Die BLOCK-START/STOP Index werden aber immer parallel zu den STRINGS mit übergeben. Wird der PATH-STRING größer 255 Zeichen so wird die PATH-Verfolgung deaktiviert, sodass nur noch „OVERFLOW“ als Text eingetragen ist.

Da bei sehr großen und komplexen XML-Daten nicht klar ist, wie lange es dauert bis der Baustein Daten zum Rückmelden findet, ist eine WATCHDOG Funktion integriert. Hiermit kann eine maximale Bearbeitungszeit parametrisiert werden. Beim Überschreiten der Zeit wird der Baustein-Aufruf automatisch abgebrochen, und im nächsten Zyklus wieder an gleicher Stelle fortgesetzt. Dabei wird der Typ-Code 98 zurückgemeldet.

Folgende Typ-Kennungen sind definiert.

Typ (Code)	Datentyp	Beschreibung
00	unbekannt	Undefiniertes Element gefunden
01	TAG (Element)	Beginn - des Element Datenzeiger für Element über BLOCK1
02	END-TAG (Element)	Ende - des Element
03	TEXT	Inhalt eines Element Datenzeiger für Value über BLOCK1
04	ATTRIBUTE	Attribute eines Element Datenzeiger für Attribute über BLOCK1 Datenzeiger für Value über BLOCK2
05	TAG (Processing Instruction)	Anweisungen für die Verarbeitung Datenzeiger für Element über BLOCK1
12	CDATA	Inhaltlich nicht analysierter TEXT Datenzeiger für Value über BLOCK1
13	COMMENT	Kommentarzeilen Datenzeiger für Value über BLOCK1
14	DTD	Dokumenten Typ Deklaration Datenzeiger für Value über BLOCK1

98	WATCHDOG	Maximale Durchlaufzeit erreicht- Abbruch
99	ENDE	Keine weiteren Elemente vorhanden

XML-Beispiel

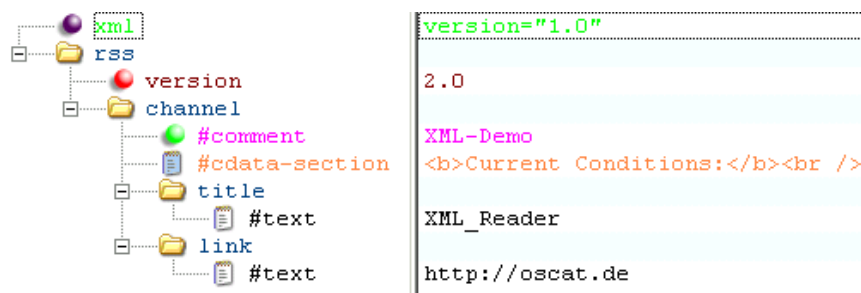
Flache Darstellung

```
<?xml version="1.0" ?><rss version="2.0"><channel><!-- XML-Demo
--><![CDATA[<b>CurrentConditions:</b><br
/>]]><title>XML_Reader</title>
<link>http://oscat.de</link></channel></rss>
```

Darstellung der Ebenen (ohne Processing-Instruction)

```
-<rss version="2.0">
  -<channel>
    <!-- XML-Demo -->
    <b>Current Conditions:</b><br />
    <title>XML_Reader</title>
    <link>http://oscat.de</link>
  </channel>
</rss>
```

Darstellung als Baumansicht mit Elementtypen



Legende:

Element		Comment	
Attribute		Processing Instruction	
Text		CDATA Section	

Anwendungs-Beispiel:

CASE STATE OF

```

00:
    STATE := 10;
    CTRL.START_POS := HTTP_GET.BODY_START; (Index des ersten Zeichen *)
    CTRL.STOP_POS  := HTTP_GET.BODY_STOP;  (Index des letzten Zeichen *)
    CTRL.COMMAND   := WORD#2#11111111_11111111; (* Init + alle Elemente melden *)
10:
    (* XML Daten seriell lesen *)
    XML_READER.CTRL := CTRL;
    XML_READER.BUF  := BUFFER;
    XML_READER();
    CTRL := XML_READER.CTRL;
    BUFFER := XML_READER.BUF;

    IF CTRL.TYP = 99 THEN
        STATE := 20; (* Exit - keine weiteren Elemente vorhanden *)
    ELSIF CTRL.TYP < 98 THEN (* bei Timeout (Code 98) nichts machen *)
        (* Auswertung der XML-Elemente über CTRL-Datenstruktur *)
    END_IF;
20:
    (* sonstiges..... *)
END_CASE;

```

Folgende Information werden dabei über die CTRL-Datenstruktur ausgegeben

```

----- erster Durchlauf -----
COUNT:          1
TYPE:             5          (OPEN ELEMENT - PROCESSING
INSTRUCTION)
LEVEL:           1
ELEMENT:         'xml'
PATH:            '/xml'
----- nächster Durchlauf -----
COUNT:          2
TYPE:             4          (ATTRIBUTE)
LEVEL:           1
ELEMENT:         'xml'
ATTRIBUTE:       'version'
VALUE:           '1.0'
PATH:            '/xml'
----- nächster Durchlauf -----
COUNT:          3
TYPE:             2          (CLOSE ELEMENT)
LEVEL:           0

```

```

ELEMENT:      'xml'
PATH:         "
----- nächster Durchlauf -----
COUNT:       4
TYPE:         1          (OPEN ELEMENT - Standard)
LEVEL:        1
ELEMENT:      'rss'
PATH:         '/rss'
----- nächster Durchlauf -----
COUNT:       5
TYPE:         4          (ATTRIBUTE)
LEVEL:        1
ELEMENT:      'rss'
ATTRIBUTE:    'version'
VALUE:        '2.0'
PATH:         '/rss'
----- nächster Durchlauf -----
COUNT:       6
TYPE:         1          (OPEN ELEMENT - Standard)
LEVEL:        2
ELEMENT:      'channel'
PATH:         '/rss/channel'
----- nächster Durchlauf -----
COUNT:       7
TYPE:         13         (COMMENT-ELEMENT)
LEVEL:        2
VALUE:        ' XML-Demo '
PATH:         '/rss/channel'
----- nächster Durchlauf -----
COUNT:       8
TYPE:         12         (CDATA)
LEVEL:        2
VALUE:        '<b>Current Conditions:</b><br />'
PATH:         '/rss/channel'
----- nächster Durchlauf -----
COUNT:       9
TYPE:         1          (OPEN ELEMENT - Standard)
LEVEL:        3
ELEMENT:      'title'
PATH:         '/rss/channel/title'
----- nächster Durchlauf -----
COUNT:       10
TYPE:         3          (TEXT)
LEVEL:        3
ELEMENT:      'title'
VALUE:        ' XML_Reader'
PATH:         '/rss/channel/title'
----- nächster Durchlauf -----

```

```

COUNT:          11
TYPE:             2          (CLOSE ELEMENT)
LEVEL:            2
ELEMENT:          'title'
PATH:             '/rss/channel'
----- nächster Durchlauf -----
COUNT:          12
TYPE:             1          (OPEN ELEMENT - Standard)
LEVEL:            3
ELEMENT:          'link'
PATH:             '/rss/channel/link'
----- nächster Durchlauf -----
COUNT:          13
TYPE:             3          (TEXT)
LEVEL:            3
ELEMENT:          'link'
VALUE:            'http://oscat.de'
PATH:             '/rss/channel/link'
----- nächster Durchlauf -----
COUNT:          14
TYPE:             2          (CLOSE ELEMENT)
LEVEL:            2
ELEMENT:          'link'
PATH:             '/rss/channel'
----- nächster Durchlauf -----
COUNT:          15
TYPE:             2          (CLOSE ELEMENT)
LEVEL:            1
ELEMENT:          'channel'
PATH:             '/rss'
----- nächster Durchlauf -----
COUNT:          16
TYPE:             2          (CLOSE ELEMENT)
LEVEL:            0
ELEMENT:          'rss'
PATH:             ''
----- nächster Durchlauf -----
COUNT:          17
TYPE:             99          (EXIT – END OF DATA)

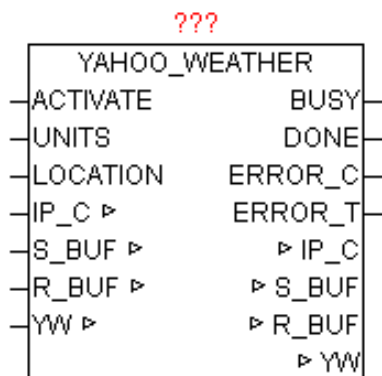
```

2.32. YAHOO_WEATHER

Type Funktionsbaustein:

IN_OUT IP_C : Datenstruktur 'IP_CONTROL' (Parametrierungsdaten)

S_BUF: Datenstruktur 'NETWORK_BUFFER' (Sendedaten)
 R_BUF: Datenstruktur 'NETWORK_BUFFER' (Empfangsdaten)
 YW: Datenstruktur 'YAHOO_WEATHER' (Wetterdaten)
 INPUT ACTIVATE: BOOL (positive Flanke startet die Abfrage)
 UNITS: BOOL (FALSE = Fahrenheit , TRUE = Celsius)
 LOCATION: STRING(20) (Ortsangabe mittels LOCATION-ID
 Code)
 OUTPUT BUSY: BOOL (Abfrage ist aktiv)
 DONE: BOOL (Abfrage ohne Fehler beendet)
 ERROR_C: DWORD (Fehlercode)
 ERROR_T: BYTE (Fehlertype)



Der Baustein lädt die aktuellen Wetterdaten des angegebenen Ortes mittels eines RSS feed (XML-Datenstruktur) von <http://weather.yahooapis.com> herunter, analysiert die XML-Daten und legt die wesentlichen Daten aufbereitet in der YAHOO_WEATHER Datenstruktur ab. Mit einer positiven Flanke von ACTIVATE wird die Abfrage gestartet und eine DNS Abfrage mit nachfolgender HTTP-GET durchgeführt. Nach erfolgreichem Empfang der Daten werden mittels XML_READER alle Elemente durchlaufen und wenn benötigt in der Datenstruktur in konvertierter Form abgelegt. Mit UNITS kann noch zwischen Fahrenheit und Celsius als Einheit gewählt werden. Durch Angabe der LOCATION_ID wird der genaue Ort des Wetters angegeben. Während die Abfrage aktiv ist, wird BUSY=TRUE ausgegeben. Nach erfolgreich beendeter Abfrage wird DONE=TRUE ausgegeben. Sollte bei der Abfrage ein Fehler auftreten so wird dieser unter ERROR_C gemeldet in Kombination mit ERROR_T.

ERROR_T:

Wert	Eigenschaften
1	Die genaue Bedeutung von ERROR_C ist beim Baustein DNS_CLIENT nachzulesen

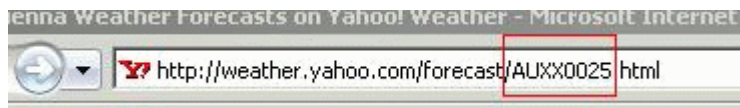
2	Die genaue Bedeutung von ERROR_C ist beim Baustein HTTP_GET nachzulesen

Suchen der Location-ID eines bestimmten Ortes:

Mittels Internet-Browser die Seite <http://weather.yahoo.com/> aufrufen, und im Eingabefeld "Enter city or zip code:" den Namen des gesuchten Ortes eingeben, und mittels "Go" suchen bzw. Aufrufen.



Nach erfolgreicher Auswahl werden im Browserfenster die aktuellen Wetterdaten des angegebenen Ortes angezeigt. In der URL (Web-Link) Zeile ist nun die Location-ID ersichtlich.



Somit ergibt der gesuchte Ort „Wien (vienna)“ die Location-ID „AUXX0025“

Beispiel eines RSS feed:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<rss version="2.0" xmlns:yweather="http://weather.yahooapis.com/ns/rss/1.0"
  xmlns:geo="http://www.w3.org/2003/01/geo/wgs84_pos#">
<channel>
  <title>Yahoo! Weather - Sunnyvale, CA</title>
  <link>http://us.rd.yahoo.com/dailynews/rss/weather/Sunnyvale__CA/
    *http://weather.yahoo.com/forecast/94089_f.html</link>
```

```

<description>Yahoo! Weather for Sunnyvale, CA</description>
<language>en-us</language>
<lastBuildDate>Tue, 29 Nov 2005 3:56 pm PST</lastBuildDate>
<ttl>60</ttl>
<yweather:location city="Sunnyvale" region="CA" country="US"></yweather:location>
<yweather:units temperature="F" distance="mi" pressure="in" speed="mph"></yweather:units>
<yweather:wind chill="57" direction="350" speed="7"></yweather:wind>
<yweather:atmosphere humidity="93" visibility="1609" pressure="30.12" rising="0"></yweather:atmosphere>
<yweather:astronomy sunrise="7:02 am" sunset="4:51 pm"></yweather:astronomy>
<image>
  <title>Yahoo! Weather</title>
  <width>142</width>
  <height>18</height>
  <link>http://weather.yahoo.com/</link>
  <url>http://us.i1.yimg.com/us.yimg.com/i/us/nws/th/main_142b.gif</url>
</image>
<item>
  <title>Conditions for Sunnyvale, CA at 3:56 pm PST</title>
  <geo:lat>37.39</geo:lat>
  <geo:long>-122.03</geo:long>
  <link>http://us.rd.yahoo.com/dailynews/rss/weather/
  <span style="font-size: 0px"> </span>Sunnyvale__CA/*
  <span style="font-size: 0px"> </span>http://weather.yahoo.com/<span style="font-size: 0px"> </span>forecast/94089_f.html
  </link>
  <pubDate>Tue, 29 Nov 2005 3:56 pm PST</pubDate>
  <yweather:condition text="Mostly Cloudy" code="26" temp="57" date="Tue, 29 Nov 2005 3:56
    pm PST"></yweather:condition>
  <description><![CDATA[
<br />
<b>Current Conditions:</b><br />
Mostly Cloudy, 57 F<p />
<b>Forecast:</b><br>
  Tue - Mostly Cloudy. High: 62 Low: 45<br />
  Wed - Mostly Cloudy. High: 60 Low: 52<br />
  Thu - Rain. High: 61 Low: 46<br />
<br />
<a href="http://us.rd.yahoo.com/dailynews/rss/weather/Sunnyvale__CA/*http://weather.yahoo.com/forecast/94089_f.html">Full
Forecast at Yahoo! Weather</a><br>
(provided by The Weather Channel)<br>]]>
  </description>
  <yweather:forecast day="Tue" date="29 Nov 2005" low="45" high="62" text="Mostly Cloudy"
    code="27"></yweather:forecast>
  <yweather:forecast day="Wed" date="30 Nov 2005" low="52" high="60" text="Mostly Cloudy"
    code="28"></yweather:forecast>

```



```

    <guid isPermaLink="false">94089_2005_11_29_15_56_PST</guid>
  </item>
</channel>
</rss>

```

Aus den XML-Daten werden folgende Elemente aufbereitet und in der YAHOO_WEATHER Datenstruktur abgelegt.

YAHOO_WEATHER Datenstruktur:

Name	Typ	Eigenschaften
TimeToLive	INT	Time to Live: how long in minutes this feed should be cached
location_city	STRING(40)	The location of this forecast: city: city name
location_region	STRING(20)	The location of this forecast: region: state, territory, or region, if given
location_country	STRING(2)	The location of this forecast: country: two-character country code
unit_temperature	STRING(1)	temperature: degree units, f for Fahrenheit or c for Celsius
unit_distance	STRING(2)	distance: units for distance, mi for miles or km for kilometers
unit_pressure	STRING(2)	pressure: units of barometric pressure, in for pounds per square inch or mb for millibars
unit_speed	STRING(3)	speed: units of speed, mph for miles per hour or kph for kilometers per hour
wind_chill	INT	Forecast information about wind chill in degrees
wind_direction	INT	Forecast information about wind direction, in degrees
wind_speed	REAL	Forecast information about wind speed, in the units (mph or kph)
atmosphere_humidity	INT	Forecast information about current atmospheric humidity: humidity, in percent
atmosphere_pressure	INT	Forecast information about current atmospheric pressure: barometric pressure, in the units (in or mb)
atmosphere_visibility	REAL	Forecast information about current atmospheric visibility, in the units (mi or km)
atmosphere_rising	INT	Forecast information about rising: state of the barometric pressure: steady (0), rising (1), or falling (2).

		(integer: 0, 1, 2)
astronomy_sunrise	STRING(10)	sunrise: today's sunrise time. The time is a string in a local time format of "h:mm am/pm"
astronomy_sunset	STRING(10)	sunset: today's sunset time. The time is a string in a local time format of "h:mm am/pm"
geo_latitude	REAL	The latitude of the location
geo_longitude	REAL	The longitude of the location
cur_conditions_temp	INT	The current weather conditions: temp: the current temperature, in the units (f or c)
cur_conditions_text	STRING(40)	The current weather conditions: text: a textual description of conditions
cur_conditions_code	INT	The current weather conditions: code: the condition code for this forecast
forecast_today_low_temp	INT	The forecast today weather conditions: the forecasted low temperature for this day in the units (f or c)
forecast_today_high_temp	INT	The forecast today weather conditions: the forecasted high temperature for this day in the units (f or c)
forecast_today_text	STRING(40)	The forecast today weather conditions: text: a textual description of conditions
forecast_today_code	INT	The current weather conditions: code: the condition code for this forecast
forecast_tomorrow_low_temp	INT	The forecast tomorrow weather conditions: the forecasted low temperature for this day in the units (f or c)
forecast_tomorrow_high_temp	INT	The forecast tomorrow weather conditions: the forecasted high temperature for this day in the units (f or c)
forecast_tomorrow_text	STRING(40)	The forecast tomorrow weather conditions: text: a textual description of conditions
forecast_tomorrow_code	INT	The current weather conditions: code: the condition code for this forecast

Condition Codes:

Die Felder cur_conditions_code, forecast_today_code und forecast_tomorrow_code beschreiben die Wetterlage in Textform durch Angabe eines „Condition Codes“

Wert	Beschreibung
------	--------------

0	tornado
1	tropical storm
2	hurricane
3	severe thunderstorms
4	thunderstorms
5	mixed rain and snow
6	mixed rain and sleet
7	mixed snow and sleet
8	freezing drizzle
9	drizzle
10	freezing rain
11	showers
12	showers
13	snow flurries
14	light snow showers
15	blowing snow
16	snow
17	hail
18	sleet
19	dust
20	foggy
21	haze
22	smoky
23	blustery
24	windy
25	cold
26	cloudy

27	mostly cloudy (night)
28	mostly cloudy (day)
29	partly cloudy (night)
30	partly cloudy (day)
31	clear (night)
32	sunny
33	fair (night)
34	fair (day)
35	mixed rain and hail
36	hot
37	isolated thunderstorms
38	scattered thunderstorms
39	scattered thunderstorms
40	scattered showers
41	heavy snow
42	scattered snow showers
43	heavy snow
44	partly cloudy
45	thundershowers
46	snow showers
47	isolated thundershowers
3200	not available

Verzeichnis der Funktionsbausteine

DNS_CLIENT.....	9	LOG_MSG.....	28
GET_WAN_IP.....	10	MB_CLIENT.....	29
HTML_DECODE.....	12	MB_SERVER.....	33
HTML_ENCODE.....	12	MB_VMAP.....	35
HTTP_GET.....	13	PRINT_SF.....	39
IP_CONTROL.....	19	READ_HTTP.....	40
IP_CONTROL2.....	27	SNTP_CLIENT.....	41
IP_FIFO.....	25	SNTP_SERVER.....	42
IP2GEO.....	17	SPIDER_ACCESS.....	43
IP4_CHECK.....	15	STRING_TO_URL.....	45
IP4_DECODE.....	16	SYS_LOG.....	46
IP4_TO_STRING.....	16	TELNET_LOG.....	49
IRTRANS_1.....	4	TELNET_PRINT.....	50
IRTRANS_4.....	5	URL_DECODE.....	54
IRTRANS_8.....	6	URL_ENCODE.....	54
IRTRANS_DECODE.....	7	URL_TO_STRING.....	54
IS_IP4.....	18	XML_READER.....	55
IS_URLCHR.....	19	YAHOO_WEATHER.....	61